ASE CHAPTER 3

T1 Multi Choice Questions Pack · SW Requirements

Section 1: Publisher Companion Slide MCQs (25 Items)

Q3.1. Which one is not a step of requirement engineering?

(a) Requirements elicitation · (b) Requirements analysis · (c) Requirements design · (d) Requirements documentation

> **(c) Requirements design**
>
> **Requirements engineering consists of four crucial process steps: requirements elicitation, requirements analysis, requirements documentation (producing the SRS), and requirements review (validation).** "Requirements design" is not a recognized phase; software design is a distinct subsequent phase of the life cycle that operates upon the frozen SRS.
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.1, p. 41; Slide 121. [Independently Checked]*

Q3.2. Requirements elicitation means:

(a) Gathering of requirements · (b) Capturing of requirements · (c) Understanding of requirements · (d) All of the above

> **(d) All of the above**
>
> **Requirements elicitation encompasses discovering, gathering, capturing, and understanding the real needs of customers, users, and organizational stakeholders.**
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.4, p. 49; Slide 121. [Independently Checked]*
>
> **ملاحظة الطالب والفخ الامتحاني:**
>
> - الاستخلاص ليس مجرد "جمع" سلبي للمعلومات، بل يشمل الاستكشاف والتوضيح والاستيعاب العميق لرغبات العميل والقيود المحيطة بالنظام.

Q3.3. SRS stands for:

(a) Software requirements specification · (b) System requirements specification · (c) Systematic requirements specifications · (d) None of the above

> **(a) Software requirements specification**
>
> **SRS stands for Software Requirements Specification.**
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.6, p. 72; Slide 122. [Independently Checked]*

Q3.4. SRS document is for:

(a) "What" of a system? · (b) How to design the system? · (c) Costing and scheduling of a system · (d) System's requirement.

(a) "What" of a system?

The SRS describes what the software must do, including required external behavior and interfaces, while leaving internal design and implementation decisions to the design stage.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.1, p. 40; Slide 122. [Independently Checked]

Q3.5. Requirements review process is carried out to:

(a) Spend time in requirements gathering · (b) Improve the quality of SRS · (c) Document the requirements · (d) None of the above

(b) Improve the quality of SRS

The requirements review process is conducted to detect and correct omissions, ambiguities, contradictions, and unrealizable requirements, thereby directly improving the technical quality and baseline integrity of the SRS.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.1.1, p. 41; Slide 123. [Independently Checked]

ملاحظة الطالب والفخ الامتحاني:

- مراجعة المتطلبات (Requirements Review) هي نشاط توكيد جودة يهدف لكشف العيوب في الـ SRS قبل الانتقال للتصميم لتفادي كلفة الإصلاح المضاعفة لاحقاً.

Q3.6. Which one of the statements is not correct during requirements engineering?

(a) Requirements are difficult to uncover · (b) Requirements are subject to change · (c) Requirements should be consistent · (d) Requirements are always precisely known.

(d) Requirements are always precisely known.

In empirical software engineering practice, requirements are rarely precisely known upfront; customers often have vague initial concepts, requirements constantly change, and discovering complete needs requires disciplined iteration.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.1, pp. 40–41; Slide 123. [Independently Checked]

Q3.7. Which one is not a type of requirements?

(a) Known requirements · (b) Unknown requirements · (c) Undreamt requirements · (d) Complex requirements

(d) Complex requirements

The textbook categorizes requirements discovery into three states: known requirements (explicitly understood and stated), unknown requirements (revealed during investigation), and undreamt requirements (innovative features beyond user expectations). "Complex requirements" is not a recognized classification category in this model.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.2, p. 45; Slide 124. [Independently Checked]

Q3.8. Which one is not a requirements elicitation technique?

(a) Interviews · (b) The use case approach · (c) FAST · (d) Data flow diagram.

(d) Data flow diagram.

Interviews, the use case approach, and FAST are requirements elicitation techniques used to capture user needs. A Data Flow Diagram (DFD) is a structural requirements analysis and data-modeling tool, not an elicitation technique.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.4, p. 49 & §3.5.1, p. 60; Slide 124. [Independently Checked]

ملاحظة الطالب والفخ الامتحاني:

- ميز بين أدوات الاستخلاص (Interviews, Use Cases, FAST, QFD) وأدوات التحليل والنمذجة الهيكلية (DFD, ERD, Data Dictionary).

Q3.9. FAST stands for:

(a) Functional Application Specification Technique · (b) Fast Application Specification Technique · (c) Facilitated Application Specification Technique · (d) None of the above

(c) Facilitated Application Specification Technique

FAST stands for Facilitated Application Specification Technique.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.4.3, p. 51; Slide 125. [Independently Checked]

ملاحظة الطالب والفخ الامتحاني:

- كلمة FAST تعني Facilitated (ميسّرة)، وهي جلسات عمل مشتركة ومكثفة بين العملاء والمطورين يقودها ميسر محايد.

Q3.10. QFD in requirement engineering stands for:

(a) Quality function design · (b) Quality factor design · (c) Quality function development · (d) Quality function deployment

(d) Quality function deployment

QFD stands for Quality Function Deployment.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.4.4, p. 53; Slide 125. [Independently Checked]

ملاحظة الطالب والفخ الامتحاني:

- احفظ المصطلح بدقة: Deployment (نشر/تطبيق وظيفة الجودة)، وترجمتها لمتطلبات هندسية ملموسة.

Q3.11. Which is not a type of requirements under quality function deployment:

(a) Normal requirements · (b) Abnormal requirements · (c) Expected requirements · (d) Exciting requirements

(b) Abnormal requirements

Under Quality Function Deployment (QFD), requirements are structured into Normal requirements (stated objectives), Expected requirements (implicit, baseline requirements), and Exciting requirements (innovative features that delight the customer). "Abnormal requirements" is not a QFD category.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.4.4, p. 53; Slide 126. [Independently Checked]

Q3.12. Use case approach was developed by:

(a) I. Jacobson and others · (b) J.D. Musa and others · (c) B. Littlewood · (d) None of the above

> **(a) I. Jacobson and others**
>
> **The Use Case Approach was developed by Ivar Jacobson and his colleagues (1992) as a scenario-driven method for capturing system behavior from the viewpoint of external actors.**
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.4.5, p. 54; Slide 126. [Independently Checked]*

Q3.13. Context diagram explains:

(a) The overview of the system · (b) The internal view of the system · (c) The entities of the system · (d) None of the above

> **(a) The overview of the system**
>
> **A context diagram represents the highest-level structural overview of the system, abstracting the entire system as a single process bubble interacting with external environment entities.**
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.5, p. 59; Slide 127. [Independently Checked]*

Q3.14. DFD stands for:

(a) Data Flow design · (b) Descriptive functional design · (c) Data flow diagram · (d) None of the above

> **(c) Data flow diagram**
>
> **DFD stands for Data Flow Diagram.**
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.5.1, p. 60; Slide 127. [Independently Checked]*

Q3.15. Level-0 DFD is similar to:

(a) Use case diagram · (b) Context diagram · (c) System diagram · (d) None of the above

> **(b) Context diagram**
>
> **In structured analysis, a Level-0 DFD is synonymous with the Context Diagram, modeling the whole target application as single process 0.**
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.5.1, p. 61; Slide 128. [Independently Checked]*

Q3.16. ERD stands for:

(a) Entity relationship diagram · (b) Exit related diagram · (c) Entity relationship design · (d) Exit related design

> **(a) Entity relationship diagram**
>
> **ERD stands for Entity Relationship Diagram.**
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.5.3, p. 63; Slide 128. [Independently Checked]*

Q3.17. Which is not a characteristic of a good SRS?

(a) Correct · (b) Complete · (c) Consistent · (d) Brief

(d) Brief

IEEE Standard 830-1998 specifies that a good SRS must be correct, unambiguous, complete, consistent, ranked for importance/stability, verifiable, modifiable, and traceable. Brevity is not a standard quality criterion; striving for shortness frequently introduces omissions and ambiguities.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.6.2, p. 73; Slide 129. [Independently Checked]

Q3.18. Outcome of requirements specification phase is:

(a) Design Document · (b) Software requirements specification · (c) Test Document · (d) None of the above

(b) Software requirements specification

The deliverable produced by the requirements specification phase is the Software Requirements Specification (SRS).

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.6, p. 72; Slide 129. [Independently Checked]

ملاحظة الطالب والفخ الامتحاني:

- وثيقة SRS هي الناتج الرسمي لمرحلة المواصفات، وهي العقد الفاصل بين متطلبات المستخدم وبداية التصميم المعماري.

Q3.19. The basic concepts of ER model are:

(a) Entity and relationship · (b) Relationships and keys · (c) Entity, effects and relationship · (d) Entity, relationship and attribute

(d) Entity, relationship and attribute

The three foundational modeling concepts of the Entity-Relationship (ER) model are Entities (data objects), Relationships (associations among entities), and Attributes (properties qualifying entities or relationships).

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.5.3, p. 63; Slide 130. [Independently Checked]

Q3.20. The DFD depicts:

(a) Flow of data · (b) Flow of control · (c) Both (a) and (b) · (d) None of the above

(a) Flow of data

A Data Flow Diagram depicts solely the flow and transformation of data through system processes; it contains no control flow, decision sequence, or loop logic.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.5.1, p. 60; Slide 130. [Independently Checked]

Q3.21. Product features are related to:

(a) Functional requirements · (b) Non functional requirements · (c) Interface requirement · (d) None of the above

(a) Functional requirements

Product features describe the operational capabilities, services, and functional transformations provided by the software to its end users, defining functional requirements.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.2.1, pp. 45–46; Slide 131. [Independently Checked]

Q3.22. Which one is a quality attribute?

(a) Reliability · (b) Availability · (c) Security · (d) All of the above

(d) All of the above

Reliability, availability, and security are examples of non-functional quality attributes.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.2.1, pp. 45–46; Slide 131. [Independently Checked]

Q3.23. IEEE standard for SRS is:

(a) IEEE Standard 837-1998 · (b) IEEE Standard 830-1998 · (c) IEEE Standard 832-1998 · (d) IEEE Standard 839-1998

(b) IEEE Standard 830-1998

The canonical standard for Software Requirements Specifications established in course literature is IEEE Standard 830-1998 (Recommended Practice for Software Requirements Specifications).

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.2.2, p. 46; Slide 132. [Independently Checked]

Q3.24. Which one is not a functional requirement?

(a) Efficiency · (b) Reliability · (c) Product features · (d) Stability

No unique answer: (a) Efficiency, (b) Reliability, and (d) Stability are all non-functional requirements.

Product features represent functional requirements, whereas efficiency, reliability, and stability are non-functional quality attributes. In this publisher slide item (Slide 132), the question contains a recognized structural flaw in its negative phrasing: three options ((a) Efficiency, (b) Reliability, (d) Stability) are non-functional requirements, while only (c) Product features is functional. If the examiner asks "*Which one is a functional requirement?*", the intended answer is (c). If asked verbatim as printed, any of (a), (b), or (d) is technically correct, with Reliability and Efficiency being canonical textbook quality attributes.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.2.1, pp. 45–46; Slide 132. [Independently Checked]

ملاحظة الطالب والفخ الامتحاني:

- **تنبيه لعيب صياغة المصدر:** هذا السؤال ورد نصاً بالسلايد 132 بصياغة ملتبسة؛ فخيارات (Efficiency, Reliability, Stability) كلها متطلبات غير وظيفية (Non-Functional / Quality Attributes)، بينما (Product features) هو الوحيد الوظيفي. إذا جاء السؤال كما هو، فإن (a) أو (b) أو (d) صحيحة بالمعنى الحرفي، أما إذا عُدلت الصياغة في الامتحان إلى "*Which one IS a functional requirement?*" فالجواب هو قطعاً (c).

MCQ-25 (Slide 133, printed as duplicate 3.23). APIs stand for:

(a) Application performance interfaces · (b) Application programming interfaces · (c) Application programming integration · (d) Application performance integration

(b) Application programming interfaces

APIs stand for Application Programming Interfaces, which establish procedural interface specifications for programmatic inter-software communication.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.2.3, p. 46; Slide 133. [Independently Checked]

ملاحظة الطالب والفخ الامتحاني:

- API = Application Programming Interface، وهو يمثل نمطاً للمواصفات الإجرائية (Procedural Interface).
  الرقم طُبع مكرراً كـ 3.23 بالسلايد 133 وثبتناه كـ MCQ-25.

Section 2: Additional Chapter 3 Synthesis MCQs (10 Items)

Q3.26. What are the three primary types of interface specifications required in software engineering?

(a) User interface, Database interface, and Network interface · (b) Procedural interfaces (APIs), Data structures, and Representation of data · (c) Hardware interface, Firmware interface, and Software interface · (d) Input interface, Output interface, and Storage interface

(b) Procedural interfaces (APIs), Data structures, and Representation of data

The three types of interface specifications are procedural interfaces, data structures, and representation of data.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.2.3, p. 46; Slide 9.

ملاحظة الطالب والفخ الامتحاني:

- التزم حرفياً بتبويب الشريحة 9 المعتمدة للدكتور: الأنواع الثلاثة لمواصفات الواجهات هي: الواجهات الإجرائية (Procedural interfaces - APIs)، هياكل البيانات (Data structures)، وتمثيل البيانات (Representation of data).
  لا تخلط بين هذا التبويب الخاص بالسلايد وبين تبويب معيار IEEE 830 الخارجي (User, Hardware, Software, Communications).

Q3.27. Under a Feasibility Study, which dimension evaluates whether the development organization has the necessary tools, platforms, and technical expertise?

(a) Operational feasibility · (b) Economic feasibility · (c) Technical feasibility · (d) Schedule feasibility

(c) Technical feasibility

Technical feasibility determines whether the proposed application can be constructed using available technology, whether technical constraints can be satisfied, and whether the development team possesses the requisite technical competencies.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.3.1, pp. 47–48; Slides 11–13.

Q3.28. What is the bounded objective of risk identification within the Feasibility Study phase according to Chapter 3 lecture materials?

(a) Executing full spiral risk cycles · (b) Determining whether major project risks are identified and can be successfully surmounted · (c) Eliminating all project uncertainties permanently · (d) Purchasing commercial insurance policies

(b) Determining whether major project risks are identified and can be successfully surmounted

Within the feasibility study, the focus is on determining whether major project risks have been identified and whether they can be surmounted before the project proceeds.

Source: Chapter 3 lecture, Slide 14; Dr. Ali Fahim Vocabulary Ledger (2026-09-30).

ملاحظة الطالب والفخ الامتحاني:

- انتبه لحدود إدارة المخاطر في هذا الفصل: ليست دورة حلزونية كاملة، بل التحقق من أن المخاطر الجوهرية محددة وقابلة للتجاوز (Surmountable) كشرط للمضي بالمشروع.

Q3.29. How do User Requirements fundamentally differ from System Requirements?

(a) User requirements are written in code; system requirements are written in English · (b) User requirements state high-level customer expectations in natural language and diagrams, while system requirements define structured, detailed functional and operational specifications for engineers · (c) User requirements describe non-functional constraints only; system requirements describe functions only · (d) User requirements are authored during testing; system requirements during elicitation

(b) User requirements state high-level customer expectations in natural language and diagrams, while system requirements define structured, detailed functional and operational specifications for engineers

User requirements are high-level statements in natural language and informal diagrams defining what services the system provides and its operational constraints for clients and management. System requirements are comprehensive, structured descriptions of software functions, services, and operational constraints serving as the contract and basis for architectural design.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), Slide 8; Sommerville (9th ed.), §4.1, pp. 84–85.

Q3.30. In Data Flow Diagramming (DFD), what constitutes a "Black Hole" defect?

(a) A process that produces outputs without receiving any inputs · (b) A process that has input data flows but generates no output data flows · (c) A data store that is never accessed · (d) A data flow connecting two external entities

(b) A process that has input data flows but generates no output data flows

In DFD rule verification, a "Black Hole" refers to a process that consumes input data flows but produces zero output data flows, violating the fundamental principle of data transformation.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.5.1, pp. 60–61; B.B. Agarwal et al. (2010), §3.3.1, p. 57.

ملاحظة الطالب والفخ الامتحاني:

- الثقب الأسود (Black Hole) في الـ DFD: عملية تدخلها بيانات دون أن يخرج منها أي تدفق. عكسها المعجزة (Miracle): عملية تنتج مخرجات دون أي مدخلات!

Q3.31. Which data flow connection is strictly illegal in a valid Data Flow Diagram?

(a) Process to Process · (b) Process to Data Store · (c) Data Store directly to another Data Store without an intervening Process · (d) External Entity to Process

(c) Data Store directly to another Data Store without an intervening Process

Data cannot move autonomously between data stores or between external entities; every valid data flow in a DFD must have a Process as either its origin or its destination.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.5.1, pp. 60–61; Slides 85–86.

Q3.32. In the course Data Dictionary notation, what does { } indicate?

(a) An optional data element · (b) Selection of one alternative from a list · (c) Repetition of a data element, with adjacent bounds specifying its permitted count · (d) Sequential composition of data elements

(c) Repetition of a data element, with adjacent bounds specifying its permitted count

In Table 3.3, x=a+b means x consists of a and b; x=[a/b] means either a or b; and x=(a) marks an optional item. Braces denote repetition: x=y{a}z means between y and z occurrences of a, while x=y{a} and x={a}z express lower-only and upper-only bounds, respectively.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.5.2, Table 3.3, p. 63; Slide 88; B.B. Agarwal et al. (2010), §3.3.2, p. 58.

ملاحظة الطالب والفخ الامتحاني:

- تنبيه مصدري: جدول 3.3 في كتاب أغروال يكتب الاختيار بين البدائل هكذا x=[a/b]، بينما سلايد 88 يستخدم {a/b}؛ اعتمدنا صيغة الكتاب لأنه المصدر الأساسي. أما التكرار المحدود في الكتاب فيظهر بصيغة x=y{a}z.

Q3.33. A requirement states: "*The software shall be intuitive, fast, and extremely user-friendly.*" Which IEEE 830 characteristic is directly violated?

(a) Modifiability · (b) Verifiability · (c) Traceability · (d) Ranking for importance

(b) Verifiability

A requirement is verifiable if and only if there exists a finite, cost-effective process by which a person or machine can measure whether the delivered software meets the requirement. Subjective adjectives like "intuitive", "fast", and "user-friendly" cannot be objectively verified or tested without quantifiable metrics (such as task completion time or maximum click count).

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.6.2, pp. 74–75; Slide 104.

Q3.34. In Use Case modeling, what is the defining distinction between an <<include>> and an <<extend>> relationship?

(a) <<include>> is optional; <<extend>> is mandatory · (b) <<include>> unconditionally executes shared common behavior, while <<extend>> conditionally adds optional behavior at defined extension points · (c) <<include>> connects actors only; <<extend>> connects use cases only · (d) They are functionally identical

(b) <<include>> unconditionally executes shared common behavior, while <<extend>> conditionally adds optional behavior at defined extension points

An <<include>> relationship represents mandatory factoring of common subroutines that execute unconditionally during the base use case. An <<extend>> relationship represents optional or exceptional behavior that is executed only if specific trigger conditions are met at declared extension points.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.4.5, p. 54; Slides 30–33.

Q3.35. What is the fundamental difference between Requirements Verification and Requirements Validation?

(a) Verification checks code syntax; Validation checks compiler optimization · (b) Verification asks "Are we building the product right?" (conformance to specification), while Validation asks "Are we building the right product?" (satisfying actual customer needs) · (c) Verification is performed exclusively by end users; Validation is performed by software developers · (d) Verification is dynamic testing; Validation is static review

(b) Verification asks "Are we building the product right?" (conformance to specification), while Validation asks "Are we building the right product?" (satisfying actual customer needs)

Verification ensures that intermediate engineering artifacts strictly adhere to preceding lifecycle specifications (conformance). Validation ensures that the specified system accurately satisfies the real-world operational intentions and operational objectives of the customer.

Source: Slide 111; B.B. Agarwal et al. (2010), §3.8, p. 68.

ASE CHAPTER 3

T2 Descriptive & Process Exercises Pack

Section 1: Core Textbook Descriptive Exercises (18 Items)

Exercise 3.1. Significance of Requirement Engineering & Problems in Requirements Formulation

Discuss the significance and use of requirement engineering. What are the problems in the formulation of requirements?

Significance and Use of Requirement Engineering: Requirement engineering (RE) is the disciplined process of discovering, analyzing, documenting, and validating the services a system must provide and the operational constraints under which it must function. Its primary significance lies in:

1. **Cost Containment:** Finding misunderstandings while requirements are being clarified can avoid carrying them into design, implementation, and later correction. No fixed cost multiplier is asserted here.
2. **Contractual Foundation:** The resulting specification serves as a **formal agreement and contract between customer and developer**, minimizing divergence between expectations and the delivered product.
3. **Architectural Baseline:** Requirements drive software architectural design, effort and cost estimation, project scheduling, verification planning, and acceptance testing.

Problems in the Formulation of Requirements:

1. **Requirements are rarely well-defined upfront:** Customers typically possess a vague concept of what they need and cannot articulate precise operational rules.
2. **Requirements volatility:** System requirements constantly change over time due to business environment dynamics, regulatory updates, and evolving user perspectives.
3. **Communication barrier:** A severe semantic gap exists between domain users (who speak organizational jargon) and software engineers (who think in data models, algorithms, and architectures).
4. **Natural language ambiguity:** Requirements written in natural language (e.g., English) inherently suffer from ambiguity, contradictions, and multiple valid interpretations.
5. **Omission of essential constraints:** Users routinely take critical implicit operational assumptions for granted, leaving vital non-functional constraints unstated until system failure occurs.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.1, pp. 40–42; companion Slide 134.

Exercise 3.3. Crucial Process Steps of Requirement Engineering

What are crucial process steps of requirement engineering? Discuss with the help of a process flow description.

Crucial Process Steps of Requirements Engineering: The requirements engineering process takes the initial Problem Statement as input, produces the Software Requirements Specification (SRS) as output, and consists of four crucial, tightly coupled iterative process steps:

1. **Requirements Elicitation:** Proactively uncovers, discovers, and gathers user expectations, operational realities, and system constraints through direct stakeholder interaction (interviews, FAST workshops, QFD, and use cases).
2. **Requirements Analysis:** Refines, models, and scrutinizes elicited requirements to resolve conflicts, remove ambiguities, and establish system boundaries (DFD leveling, Data Dictionary, ER modeling).
3. **Requirements Documentation:** Formalizes analyzed requirements into a standardized, clear, and unambiguous technical contract—the **Software Requirements Specification (SRS)** conforming to IEEE Std 830.
4. **Requirements Review (Validation):** Rigorously inspects the draft SRS through formal peer walkthroughs and quality checklists to detect defects before implementation, feeding iterative rework loops back to Documentation and Elicitation.

The Role of Feasibility Study: *Feasibility Study* is an early, pre-project evaluation filter that precedes full requirements engineering. It evaluates technical, economic, and operational viability to decide whether to proceed to RE, re-scope constraints, or terminate the project before capital is committed.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.1.1, pp. 41–42; companion Slide 3 & Slide 134.

ملاحظة أكاديمية هامة (Academic Note)

فخ الامتحان (Exam Trap): إذا طُلب رسم أو شرح خطوات RE حسب مخطط د. علي فهيم المعتمد (Figure 3.1 والسلايد 3)، فالخطوات الأربع الأساسية هي: الاستخلاص (Elicitation) ← التحليل (Analysis) ← التوثيق (Documentation/SRS) ← المراجعة (Review/Validation) مع حلقتي تغذية راجعة (Rework Loops). أما دراسة الجدوى (Feasibility Study) فهي بوابة تقييم سابقة لتحديد جدوى الدخول في هندسة المتطلبات من الأساس وليست خطوة خامسة داخل حلقة التوصيف.

Exercise 3.4. Present State of Practice in Requirement Engineering & Improvement Steps

Discuss the present state of practices in requirement engineering. Suggest few steps to improve the present state of practice.

Present State of Practice in Requirement Engineering: Recurring difficulties in requirements practice include:

1. **Requirements are difficult to uncover:** Stakeholders may not initially know or clearly express all their needs.
2. **Requirements change:** New information and changing circumstances can alter requirements.

3. **There is no single best technique:** CASE tools can help, but they do not replace sound requirements-engineering methods and processes.
4. **Project schedules may be too tight:** Insufficient time can leave requirements analysis incomplete.
5. **Communication barriers arise:** Customers and developers may use different vocabularies and have different backgrounds.
6. **Software may be market-driven:** Products may need to serve anonymous or changing customers and keep evolving through upgrades.
7. **Resources may be limited:** A project may not have enough resources to satisfy every customer request, so requirements must be prioritized.

Practical improvements (synthesis):

- Involve representative stakeholders and use suitable elicitation techniques; clarify and review requirements iteratively.
- Prioritize requests against available time and resources, and allow for change, especially in market-driven products.
- Reserve enough time for analysis and document agreed requirements clearly in an SRS.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.1.2, pp. 42–43; companion Slides 4–5.

ملاحظة أكاديمية هامة (Academic Note)

الخلاصة: بعض التحديات تتعلق بفهم الاحتياجات والتواصل، وأخرى بالوقت والسوق والموارد؛ لذلك يلزم إشراك المعنيين وتوضيح المتطلبات ومراجعتها وترتيب الأولويات.

Exercise 3.5. Importance of Requirements & Major Requirement Categories

Explain the importance of requirements. How many types of requirements are possible and why?

Importance of Requirements: Requirements constitute the absolute foundation of the entire software engineering lifecycle. A flawed requirement inevitably leads to an incorrect architectural design, defective implementation, and invalid verification tests. A precise requirements specification enables accurate resource, cost, and schedule estimations, forms the baseline for project validation and customer sign-off, and prevents costly late-stage redesign.

There is no single count across all requirement classifications. The principal distinctions are separate axes:

1. **Functional and Non-Functional Requirements:** This is the main distinction between what services the system provides and constraints on those services or the system.
2. **User and System Requirements:** This is a separate distinction by level of detail and intended reader; user requirements are higher-level, while system requirements are more detailed.
3. **Interface Requirements:** These specify interactions with external systems, hardware, users, and communications interfaces; they are a category of requirements, not another name for user/system requirements.
4. **Known, Undreamed, and Unknown Requirements:** Requirements can also be classified by how well stakeholders can anticipate or recognize them. This is a different classification axis.

Thus, if “types” refers to the principal functional classification, answer **two: functional and non-functional**. The other pairs above classify requirements from different viewpoints.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.2.1–§3.2.2, pp. 45–47; companion Slides 6–9 & Slide 134.

ملاحظة أكاديمية هامة (Academic Note)

التمييز الأساسي: التصنيف الوظيفي/non-functional يختلف عن تصنيف user/system؛ الأول يصف طبيعة المتطلب، والثاني يصف مستوى التفصيل والجمهور المقصود.

Exercise 3.6. Systematic Steps of Requirements Engineering & Essentiality

Describe the various steps of requirements engineering. Is it essential to follow these steps?

Systematic Steps of Requirements Engineering: Requirements engineering comprises four tightly coupled, iterative core activities:

1. **Requirements Elicitation:** Discovering stakeholder needs through suitable techniques such as interviews, FAST, QFD, and use cases.
2. **Requirements Analysis & Modeling:** Refining elicited statements, resolving contradictory demands, and building formal models (Data Flow Diagrams, Data Dictionaries, Entity-Relationship Diagrams) to define functional scope.
3. **Requirements Documentation (SRS):** Authoring the formal, unambiguous baseline document conforming to IEEE Std 830-1998, capturing functional, interface, performance, and operational constraints as a black-box contract.
4. **Requirements Review & Validation:** Rigorously inspecting the draft SRS through structured peer walkthroughs and 7-point quality checklists (redundancy, completeness, ambiguity, consistency) to eliminate defects before design and coding.

These activities need not form a rigid one-way sequence. Requirements review is continuous and integrated with elicitation, analysis, and documentation.

Is It Essential to Follow These Steps? The activities are all important, but they can be revisited as requirements change. Their purpose is to improve understanding and the SRS; they do not have to follow a rigid sequence, and omissions can have different effects.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.1.1, pp. 41–42; companion Slide 3 & Slide 134.

ملاحظة أكاديمية هامة (Academic Note)

مهم: احفظ الأنشطة الأربع، لكن تذكّر أن مراجعة المتطلبات عملية مستمرة وليست مرحلة جامدة لا تتكرر.

Exercise 3.7. Requirements Elicitation Definition & Two Primary Techniques

What do you understand with the term "requirements elicitation"? Discuss any two techniques in detail.

Definition of Requirements Elicitation: Requirements elicitation is the practice of discovering, collecting, and uncovering the needs, desires, operational realities, and system constraints from stakeholders

through proactive collaborative inquiry. It is an active exploratory process rather than passive collection, because stakeholders rarely know their complete requirements in advance.

Technique 1: Structured Interviews:

- **Mechanism:** The analyst meets face-to-face with individual stakeholders (users, managers, domain experts) using open-ended questions (to uncover broad goals) or closed-ended questions (to confirm specific facts).
- **Advantages:** Builds personal rapport, yields deep contextual insights, reveals informal workarounds, and clarifies ambiguous operations.
- **Limitations:** Extremely time-consuming; interviewees may omit crucial tacit knowledge; statements can be subjective and contradictory across different organizational levels.

Technique 2: Facilitated Application Specification Technique (FAST):

- **Mechanism:** A joint collaborative workshop bringing developers and representative users together in an intensive neutral setting facilitated by an objective coordinator. Participants create individual candidate lists of objects, services, and constraints, merge and discuss them on wall charts, eliminate duplicates, prioritize items, and draft preliminary specifications.
- **Advantages:** Breaks organizational communication barriers, accelerates consensus, eliminates cross-departmental blame, and produces agreed-upon preliminary requirements within days.
- **Limitations:** Requires strict facilitator control; high organizational cost to gather all key stakeholders; can be dominated by aggressive personalities if not managed properly.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.4.1 & §3.4.3, pp. 49–53; companion Slides 16–26 & Slide 134.

ملاحظة أكاديمية هامة (Academic Note)

مقارنة امتحانية سريعة: المقابلات (Interviews) تركز على الرؤية الفردية المتعمقة لكل مستخدم، بينما ورش العمل المشتركة (FAST) تركز على بناء إجماع جماعي سريع بين المستخدمين والمطورين في جلسة واحدة.

Exercise 3.8. Inventory of Elicitation Techniques & The Most Popular Choice

List out requirements elicitation techniques. Which one is most popular and why?

Inventory of Requirements Elicitation Techniques:

1. **Interviews (Structured, Unstructured, and Semi-Structured)**
2. **Brainstorming sessions**
3. **Facilitated Application Specification Technique (FAST)**
4. **Quality Function Deployment (QFD)**
5. **Use Case Approach**
6. **Prototyping**

Most popular: Interviews. They let the analyst ask questions directly, clarify answers, and learn about users' work and expectations. No single technique is sufficient for every situation.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.4.1, pp. 49–50; companion Slide 134.

> 📝 **ملاحظة أكاديمية هامة (Academic Note)**
> **تنبيه:** لا تخلط FAST مع JAD؛ أجب باسم التقنية كما ورد في المحاضرة والكتاب.

Exercise 3.10. Quality Function Deployment (QFD) & Requirement Value Factors

Discuss quality function deployment technique of requirements elicitation. Why an importance or value factor is associated with every requirement?

Quality Function Deployment (QFD) Technique: QFD helps translate customer requirements into technical requirements. Customer requirements are grouped as normal, expected, or exciting, and each requirement is assigned an importance value.

Three Core Requirement Types in QFD:

1. **Normal Requirements:** The customer states these directly; satisfying them meets an expressed need.
2. **Expected Requirements:** The customer may not state these because they are assumed; their absence can cause dissatisfaction.
3. **Exciting Requirements:** These go beyond stated expectations and can increase satisfaction when present.

Why an Importance or Value Factor Is Associated with Every Requirement: An importance scale helps compare requirements and set priorities. The scale ranges from **1 (unrealistic; further exploration needed)** to **5 (very important)**. Requirements may then be considered achievable, deferred with a reason, or impossible and dropped; achievable requirements are implemented according to their assigned priority.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.4.4, pp. 52–54; companion Slide 135.

> 📝 **ملاحظة أكاديمية هامة (Academic Note)**
> **فخ د. علي فاهم (Exam Trap):** تذكر دائماً تصنيف QFD الثلاثي:
>
> 1. Normal: يُطلب صراحة.
> 2. Expected: غالباً لا يُطلب صراحةً؛ غيابه يسبب عدم رضا العميل.
> 3. Exciting: ميزة مبتكرة مبهرة غير متوقعة (غيابها لا يضر ووجودها يبهر العميل).

Exercise 3.11. Use Case Approach to Elicitation & Operational Guidelines

Explain the use case approach of requirements elicitation. What are use-case guidelines?

The Use Case Approach to Requirements Elicitation: Introduced by Ivar Jacobson, the use case approach is a scenario-based elicitation and behavioral modeling technique that captures how external actors interact with the system to achieve a specific measurable goal.

- An **Actor** represents an external entity (human role, external hardware device, or outside software application) that interacts with the system.

- A **Use Case** represents a complete sequence of transactions initiated by an actor that yields an observable result of value.

Core Use-Case Authoring Guidelines:

1. **Identify System Boundaries:** Explicitly define what is internal to the software system and what remains external in the operating environment.
2. **Identify Primary and Secondary Actors:** Determine all external roles that initiate system actions or receive outputs.
3. **Focus on User Goals, Not Internal Algorithms:** Name each use case with an active verb-noun phrase representing a user goal (e.g., Issue Library Card, Generate Inward Bill), completely omitting internal processing details.
4. **Specify the Happy Path First:** Clearly document the basic flow of events (main success scenario) before identifying variations.
5. **Document Alternative and Exception Flows:** Explicitly account for failure branches, missing permissions, invalid inputs, and recovery mechanisms.
6. **Avoid Implementation Detail:** Do not specify GUI buttons, database queries, programming languages, or internal data structures in use case descriptions.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.4.5, pp. 55–59; companion Slides 27–35 & Slide 135.

📝 **ملاحظة أكاديمية هامة (Academic Note)**

قاعدة ذهبية في كتابة الـUse Cases: ابدأ دائماً بصيغة فـعـل + اسم تمثل غاية المستخدم وليس شاشات النظام؛ مثلاً اكتب Withdraw Cash ولا تكتب Enter ATM Screen and Click OK Button.

Exercise 3.12. Components of a Use Case Diagram & Operational Usage

What are components of a use case diagram? Explain their usage with the help of an example.

Components of a Use Case Diagram:

1. **System Boundary:** A rectangular box enclosing the functional scope of the application; anything inside represents system functionality; anything outside represents the environment.
2. **Actors:** Depicted as stick figures placed outside the system boundary, representing external roles interacting with the system.
3. **Use Cases:** Depicted as horizontal ellipses inside the boundary containing the name of the functional goal.
4. **Communication Associations:** Solid lines connecting actors to the use cases they initiate or participate in.
5. **Stereotype Relationships:**
   - **<<include>>:** Unconditional inclusion of a shared sub-process (mandatory dependency).
   - **<<extend>>:** Conditional insertion of optional behavior triggered only under specific extension point conditions.

Illustrative Example (Banking ATM System):

- **Actor:** Customer, Bank Consortium Server.
- **System Boundary:** Automated Teller Machine (ATM).
- **Use Cases:** Withdraw Cash, Check Balance, Transfer Funds.
- **Usage & Inclusion:** Withdraw Cash has an <<include>> relationship pointing to Authenticate User PIN (every cash withdrawal unconditionally requires PIN authentication).
- **Extension:** Dispense Overdraft Warning points to Withdraw Cash with an <<extend>> relationship, executing only if the account balance drops below the threshold.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.4.5, pp. 58–59; companion Slides 36–50 & Slide 135.

ملاحظة أكاديمية هامة (Academic Note)

قاعدة السهم في علاقات Use Case: في علاقة <<include>> السهم ينطلق من الحالة الأصلية باتجاه الحالة المشتركة الإلزامية (Verify PIN → Withdraw Cash). أما في <<extend>> السهم ينطلق من الحالة الإضافية الاختيارية عائداً إلى الحالة الأساسية (Withdraw Cash → Overdraft Warning).

Exercise 3.17. Degree of a Relationship in ER Modeling with Concrete Examples

What is the degree of a relationship? Give an example of each of the relationship degree.

Definition of Degree of a Relationship: The degree of a relationship is the number of entity types that participate in that relationship instance.

Relationship Degrees and Concrete Examples:

1. **Unary (Recursive) Relationship (Degree 1):**
   - **Definition:** An association between occurrences of the same single entity type.
   - **Example:** An EMPLOYEE entity where an employee manages other employees (Manages relationship linking EMPLOYEE to EMPLOYEE as Supervisor and Subordinate).
2. **Binary Relationship (Degree 2):**
   - **Definition:** An association between occurrences of two distinct entity types. This is by far the most common relationship in data modeling.
   - **Example:** STUDENT enrolls in COURSE.
3. **Ternary Relationship (Degree 3):**
   - **Definition:** An association simultaneously linking occurrences of three distinct entity types.
   - **Example:** DOCTOR prescribes MEDICATION to a PATIENT. All three entities are required to uniquely define the prescription event.
4. **N-ary Relationship (Degree $N \geq 4$):**
   - **Definition:** An association involving N distinct entity types simultaneously.
   - **Example:** In a complex university logistics system: PROFESSOR, COURSE, CLASSROOM, and TIME_SLOT combined into a single 4-way scheduling association.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.5.3, pp. 66–68; companion Slides 87–91 & Slide 136.

> **ملاحظة أكاديمية هامة (Academic Note)**
> **تنبيه مفاهيمي:** درجة العلاقة (Degree) تعني *عدد الكيانات المشتركة* (Unary, Binary, Ternary)، بينما التعددية (Cardinality) تعني *نسبة عدد مرات الارتباط بين السجلات* (1:1, 1:N, M:N). لا تخلط بين المفهومين!

Exercise 3.22. Short Technical Notes on DFD and Data Dictionary

Write short notes on: (i) Data flow diagram, (ii) Data dictionary.

(i) Data Flow Diagram (DFD): A Data Flow Diagram is a graphical, hierarchical modeling tool that depicts the flow of data through an information system and the transformation processes applied to that data.

- **Core Elements:**
  1. **Process (Circle/Bubble):** Transforms incoming data flows into outgoing data flows.
  2. **Data Flow (Named Arrow):** Represents data packets in motion.
  3. **Data Store (Open/Parallel Lines):** Represents data at rest (files, databases).
  4. **External Entity (Rectangle / Source & Sink):** Represents outside producers or consumers of information.
- **Hierarchical Leveling:** Begins with a high-level **Context Diagram (Level 0)** establishing the whole system boundary, which decomposes into **Level 1**, **Level 2**, and detailed primitive functional processes.

(ii) Data Dictionary (DD): A Data Dictionary is a structured, centralized repository containing formal definitions, descriptions, composition schemas, and representations of all data items, flows, and stores present in the system.

- **Significance:** Acts as the single source of truth, eliminating naming ambiguities, ensuring cross-team consistency, and verifying that every data flow in a DFD is mathematically and structurally accounted for.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.5.1 & §3.5.2, pp. 60–63; companion Slides 82–86 & Slide 137.

> **ملاحظة أكاديمية هامة (Academic Note)**
> **التكامل الوظيفي:** الـDFD يعرض مسار وتحويل البيانات رسومياً، بينما القاموس (Data Dictionary) هو المرجع المكتوب الذي يشرح بالتفصيل التركيب الدقيق لكل سهم ومخزن يظهر في ذلك المخطط.

Exercise 3.25. Throw-Away Prototypes in Large Software Systems

Explain why, for large software systems development, is it recommended that prototypes should be "throw-away" prototypes?

Role of Throw-Away Prototypes in Large Systems: A throw-away prototype can help users and developers clarify requirements before committing to the final system. **It is built quickly for analysis and user feedback, then discarded; the final system is built separately.** This is useful when the prototype is not intended to become the product.

> A prototype intended to evolve into the final product follows a different approach: it must be developed and maintained with the quality expected of that product. Thus, "throw-away" applies to the prototype artifact, not to the knowledge gained from it.
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.5.4, pp. 68–71; companion Slide 116 & Slide 137.*

> 📝 **ملاحظة أكاديمية هامة (Academic Note)**
>
> **تذكّر:** في النموذج الرامي للتخلص منه تُحفظ نتائج التعلّم والمتطلبات، بينما يُترك النموذج البرمجي نفسه.

Exercise 3.30. Additional Desirable Characteristics of an SRS

Are there other characteristics of an SRS (besides listed in section 3.4.2) that are desirable? List a few and describe why.

> **Common SRS Quality Characteristics:** *Correct, Unambiguous, Complete, Consistent, Ranked for Importance/Stability, Verifiable, Modifiable, Traceable.*
>
> **Additional Desirable Characteristics:**
>
> 1. **Design Independence (Implementation Freedom):**
>   - The SRS must specify **what** the system must do without prescribing **how** it should be implemented. It should avoid dictating programming languages, internal database schemas, or specific component frameworks unless explicitly mandated by external client constraints.
> 2. **Maintainability and Expandability:**
>   - The SRS should be structured modularly with clear section numbering and cross-references so that future enhancements, regulatory changes, or system extensions can be integrated with minimal disruption to existing text.
> 3. **Usability by Multiple Stakeholders (Readability):**
>   - The document must be comprehensible to diverse audiences: clients and end-users must understand it to confirm business intent, while designers, programmers, and testers need precise technical criteria for implementation and verification.
> 4. **Conciseness and Non-Redundancy:**
>   - Information should be stated precisely once. Redundant statements of the same requirement introduce inconsistency risk whenever modifications occur.
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.6.2, pp. 73–75; companion Slide 103–105 & Slide 139.*

> 📝 **ملاحظة أكاديمية هامة (Academic Note)**
>
> **ملاحظة مصدرية:** إحالة نص التمرين إلى §3.4.2 لا تطابق موضوع الصفحات 56–57 في النسخة الممسوحة؛ اعتمدت خصائص الـSRS من موضعها في §3.6.2، ص.73–75.
>
> **أهم ميزة إضافية:** استقلالية التصميم (Design Independence)؛ وثيقة المتطلبات السيئة هي التي تتحول إلى كراس تصميم يملي على المبرمج أسماء الجداول ولغة البرمجة بدلاً من وصف المشكلة الحقيقية.

Exercise 3.31. Definition of SRS, Advantages of Standards, & Black-Box Concept

What is software requirements specification (SRS)? List out the advantages of SRS standards. Why is SRS known as the black box specification of a system?

Definition of Software Requirements Specification (SRS): The SRS is the official, comprehensive document that completely describes the behavior, functional capabilities, performance parameters, design constraints, and external interfaces of a software system. It establishes the formal baseline agreement between customers and developers.

Advantages of SRS Standards (e.g., IEEE Std 830):

1. **Completeness Framework:** Standard templates ensure that analysts do not forget critical aspects such as security, error handling, performance limits, and hardware interfaces.
2. **Cross-Project Consistency:** Standardized organization enables developers, testers, and reviewers to locate specific information rapidly regardless of who authored the document.
3. **Facilitated Communication:** Establishes a common professional vocabulary and unambiguous structure across diverse teams and contractors.
4. **Streamlined Quality Auditing:** Allows quality assurance teams to verify that mandatory contractual and regulatory clauses are satisfied.

Why SRS Is Known as a "Black-Box" Specification: An SRS treats the software system strictly as an opaque black box: it specifies all external stimuli (inputs), all observable responses (outputs), and operational performance constraints **without exposing or prescribing the internal implementation mechanisms, internal data structures, or algorithmic logic.**

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.6, pp. 72–73; companion Slide 101–103 & Slide 139.

ملاحظة أكاديمية هامة (Academic Note)

مفهوم الصندوق الأسود (Black-Box): الـSRS يوثق ما يراه المستخدم من الخارج فقط (المدخلات والمخرجات وسلوك النظام)، ولا يتدخل بما يدور داخل المحرك البرمجي من خوارزميات أو تراكيب بيانات داخلية.

Exercise 3.32. Model of a Data Dictionary, Contents, & Advantages

State the model of a data dictionary and its contents. What are its advantages?

Model of a Data Dictionary: A data dictionary defines data items and composite structures using the following notation:

- `x = a + b` : **x consists of data elements a and b**
- `x = [a|b]` : **x consists of either data element a or b**
- `x = a` : **x consists of an optional data element a**
- `x = y{a}` : **x consists of y or more occurrences of data element a**
- `x = {a}z` : **x consists of z or fewer occurrences of data element a**
- `x = y{a}z` : **x consists of some occurrences of a, between y and z**

Contents of a Data Dictionary Entry:

1. **Name of the data item**
2. **Aliases** (other names for the item)
3. **Description / purpose**
4. **Related data items**
5. **Range of values**
6. **Data structure definition / form**

Uses of a Data Dictionary: It can be used to **create ordered lists of all data items or a subset, find a data-item name from its description, and design screens and test cases**.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.5.2, pp. 62–63; companion Slides 85–86 & Slide 139.

ملاحظة أكاديمية هامة (Academic Note)

ترميز القاموس: `x = a + b` للتركيب، `x = [a|b]` للاختيار، `x = a` للعنصر الاختياري، و`x = y{a}z` لتحديد التكرار بين حدّين.

Exercise 3.33. Desirable SRS Characteristics, Formal Specs Advantages, & Core Issues

List five desirable characteristics of a good SRS document. Discuss the relative advantages of formal requirement specifications. List the important issues, which an SRS must address.

Five Desirable Characteristics of a Good SRS:

1. **Correct:** Every requirement stated reflects a true capability or constraint desired by the client.
2. **Unambiguous:** Every requirement statement has exactly one interpretation.
3. **Complete:** Includes all significant requirements, responses to all inputs (valid and invalid), and full definitions.
4. **Consistent:** No two statements conflict regarding functionality, terminology, or timing.
5. **Verifiable:** There exists a finite, cost-effective process by which a person or machine can prove the software meets the requirement.

Relative Advantages of Formal Requirement Specifications:

- **Mathematical Precision:** Expressing requirements in mathematical notations (e.g., Z, VDM) completely eliminates natural language ambiguity and subjective interpretation.
- **Automated Verification:** Allows mathematical theorem provers and automated tools to verify consistency and completeness before coding.
- **Disadvantage / Trade-off:** High training barrier; extremely difficult for non-technical clients to read and validate.

Core Issues an SRS Must Address:

1. **Functionality:** What the software is to do.
2. **External Interfaces:** Interactions with people, hardware, and other software.
3. **Performance:** Speed, throughput, memory, and timing constraints.
4. **Attributes (Quality):** Reliability, availability, security, and maintainability.
5. **Design Constraints:** Operating system requirements, standards compliance, and resource limits.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.6.1 & §3.6.2, pp. 72–75; companion Slide 101–105 & Slide 139.

ملاحظة أكاديمية هامة (Academic Note)

موازنة المواصفات الرسمية: المواصفات الرياضية (Formal Specifications) تلغي اللبس تماماً وتتيح الفحص الآلي، لكن عيبها القاتل هو عجز العميل العادي عن قراءتها، مما يصعب توقيع العقد والمصادقة عليها.

Exercise 3.35. Organization of an SRS According to IEEE Std 830 & Key Issues

Discuss the organization of a SRS. List out some important issues of this organization.

Standard Organization of an SRS (IEEE Std 830-1998 Framework):

1. **Introduction:**
   - 1.1 Purpose (intended audience and scope of document)
   - 1.2 Scope (software identification, benefits, and objectives)
   - 1.3 Definitions, Acronyms, and Abbreviations
   - 1.4 References (prior documents, standards, manuals)
   - 1.5 Overview (organization of remaining document)
2. **Overall Description:**
   - 2.1 Product Perspective (independent product or subsystem component)
   - 2.2 Product Functions (high-level functional summary)
   - 2.3 User Characteristics (educational level, technical competence)
   - 2.4 General Constraints (regulatory rules, hardware bounds)
   - 2.5 Assumptions and Dependencies (third-party factors)
3. **Specific Requirements:**
   - Detailed, verifiable requirements organized by system mode, user class, object, or functional hierarchy. Covers:
     - External Interface Requirements (User, Hardware, Software, Communications)
     - Functional Requirements (detailed inputs, processing, outputs)
     - Performance Requirements (speed, memory, concurrent users)
     - Design Constraints (standards, platform limitations)
     - Software System Quality Attributes (Reliability, Availability, Security, Maintainability)

Important Organizational Issues:

1. **Avoiding Implementation Infiltration:** Keeping section 3 focused strictly on problem specification rather than architectural solution.
2. **Structuring for Change:** Organizing functional requirements modularly so modifications affect localized sections without creating widespread ripple effects.
3. **Maintaining Traceability:** Ensuring every specific requirement has a unique hierarchical tag (e.g., FR-01.2) to support bidirectional tracing.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.6.3, pp. 75–78; companion Slide 106–110 & Slide 139.

> 📝 **ملاحظة أكاديمية هامة (Academic Note)**
>
> **الهيكل الثلاثي لمعيار IEEE 830:**
>
> 1. Introduction: تمهيد وتعريفات ومراجع.
> 2. Overall Description: نظرة عامة شاملة بلا تفاصيل تقنية معقدة.
> 3. Specific Requirements: المتطلبات التفصيلية القابلة للفحص والتحقق للمبرمجين والمختبرين.

Synthesis 19. Feasibility Study: Purpose, Dimensions, & Bounded Risk Management

What is the purpose and focus of a feasibility study in requirements engineering? Discuss its core dimensions, bounded risk management objectives, and the resulting deliverable.

Purpose and Focus of a Feasibility Study: A feasibility study is a short, focused assessment conducted at the earliest stage of requirements engineering to evaluate the potential impact, viability, and practicality of a proposed software project.

- **Primary Objective:** Determine whether the project concept is viable and whether development should proceed, be re-scoped, or be terminated before significant financial capital and engineering effort are expended.
- **Historical Project Figures:** A historical IBM report gave estimates of **31% of projects cancelled, 53% over budget by an average of 189%, and 94 restarts per 100 projects**. Treat these as historical estimates, not current universal rates. Early feasibility assessment examines viability, realistic costs and schedules, and major risks before further resources are committed.

Core Dimensions of Feasibility Assessment:

A feasibility study considers whether the project is technically possible, whether cost and schedule estimates are realistic, and whether the project is justified and workable for its intended users and organization.

Feasibility Study Focus Checklist:

- Is the product concept viable, and can a product matching the project vision be developed?
- Are the estimated cost and schedule realistic, and how large is the gap from the original estimates?
- Is the business model justified by the estimated costs and benefits?
- Have major project risks been identified and can they be surmounted?
- Are specifications complete and stable enough to support development?
- Can users and developers agree on a detailed user-interface prototype?
- Is the development plan complete and adequate?

Risk-management scope: At this stage, identify the major project risks and assess whether they can be surmounted; this is not a full risk-management lifecycle.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.3, pp. 47–49; companion Slides 10–15; Dr. Ali Fahim Vocabulary Heading 3.

> 📝 **ملاحظة أكاديمية هامة (Academic Note)**
>
> **التأطير المنهجي لدراسة الجدوى:** دراسة الجدوى ليست كتابة متطلبات تفصيلية، بل هي دراسة استكشافية للإجابة عن سؤال استراتيجي: *هل يستحق هذا المشروع الاستثمار وهل يمكن بناؤه تقنياً واقتصادياً؟* وإلغاء المشروع غير المجدي في هذه المرحلة هو نجاح هندسي يحمي الشركة من خسائر فادحة.

Synthesis 20. The Three Interface Specification Types in Software Requirements

Explain the role of interface specifications in software requirements engineering. Detail the three primary interface types highlighted in companion Slide 9 and the external interface requirements mandated by IEEE Std 830.

Role of Interface Specifications: Software systems rarely operate in complete isolation; they interact continuously with users, physical hardware devices, external databases, and third-party software services. **Interface specification defines the exact syntactic, semantic, and communication boundaries governing these interactions.** Precise interface documentation is critical for customers, system integrators, and independent module developers.

The Three Primary Interface Types:

1. **Procedural Interfaces (Application Programming Interfaces - APIs):**
   - Define the programmatic mechanism through which external software components invoke system services.
   - **Specifications include:** Function/method signatures, input parameter types, return data types, exception handling behaviors, and preconditions/postconditions.
2. **Data Structures:**
   - Define the internal and shared organization of complex information exchanged across components or stored in shared memory areas (e.g., records, queues, linked buffers).
3. **Representation of Data:**
   - Defines the physical and encoding format of data streams exchanged across network interfaces or file systems (e.g., JSON schemas, XML schemas, IEEE floating-point standards, binary packet headers).

External Interface Classification (IEEE Std 830): In a complete SRS document, interface specifications are formally partitioned into:

- **User Interfaces (UI):** Screen layouts, input styles, menu structures, keyboard shortcuts, and accessibility rules.
- **Hardware Interfaces:** Physical device addresses, baud rates, communication buses, memory-mapped I/O, and hardware interrupt signals.
- **Software Interfaces:** Operating system services, database management system (DBMS) connections, and external third-party software libraries.
- **Communications Interfaces:** Network protocol suites (TCP/IP, HTTP/REST, WebSockets), port numbers, encryption standards (TLS), and message buffering.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.2.1, pp. 45–46; companion Slide 9; IEEE Std 830-1998 §3.1.

📝 **ملاحظة أكاديمية هامة (Academic Note)**

تنبيه امتحاني دقيق: ميز بين التصنيف المنهجي في سلايد 9 (APIs, Data structures, Representation of data) وبين تبويب معيار IEEE 830 (User, Hardware, Software, Communications Interfaces). كلاهما يكمل الآخر لتغطية شاملة.

Synthesis 21. Requirements Review & Validation Process: Stages, Actions, & Checklist

Describe the systematic Requirements Review Process. Detail its six sequential stages, problem resolution actions, and the 7-point review checklist.

The Requirements Review Process: Requirements validation is the final phase of requirements engineering aimed at ensuring that the SRS accurately and completely specifies stakeholder intentions. **The formal review process is a multi-disciplinary inspection where analysts, domain experts, customers, and developers evaluate the specification against established quality criteria.**

Six Sequential Stages of the Review Process:

1. **Plan Review:** Define the review scope, schedule the session, establish ground rules, and select the multi-disciplinary inspection team.
2. **Distribute SRS Documents:** Deliver the draft SRS specification and review guidelines to all participants well in advance of the meeting.
3. **Read Documents (Individual Preparation):** Reviewers individually inspect the document, annotate anomalies, formulate queries, and check conformance against standards.
4. **Organize Review (Inspection Meeting):** The team convenes in a structured session to walk through the document systematically, logging all identified defects and ambiguities.
5. **Follow-Up Actions:** The review leader coordinates and tracks the agreed follow-up actions arising from the review.
6. **Revise Document:** The requirements author updates the SRS to incorporate the agreed corrections.

Problem Resolution Actions:

- **Requirements Clarification:** Clarify vague or ambiguous phrasing.
- **Missing Information:** Re-engage stakeholders to retrieve omitted business rules.
- **Requirements Conflicts:** Convene stakeholder negotiations to resolve contradictory demands.
- **Unrealistic Requirements:** Consult stakeholders to scale back infeasible scope.
- **Security Issues:** Verify and align system requirements with institutional security standards.

The 7-Point Quality Review Checklist:

1. **Redundancy:** Is the same requirement unnecessarily stated multiple times?
2. **Completeness:** Are all functions, states, inputs, and outputs defined?
3. **Ambiguity:** Can any requirement statement be interpreted in multiple ways?
4. **Consistency:** Are there conflicting constraints or mutual contradictions?
5. **Organization:** Does the document strictly conform to the established SRS standard?
6. **Conformance:** Does the specification adhere to organizational and legal standards?
7. **Traceability:** Is each requirement uniquely tagged and bidirectionally traceable?

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.7, pp. 92–94; companion Slides 112–115.

ملاحظة أكاديمية هامة (Academic Note)

قائمة الفحص السبعة (The 7 Checkpoints): احفظ نقاط التحقق السبعة لعملية المراجعة: Redundancy (عدم التكرار)، Completeness (الشمول)، Ambiguity (الوضوح)، Consistency (التناسق)، Organization (الترتيب)، Conformance (المطابقة)، Traceability (القابلية للتتبع).

ASE CHAPTER 3

T3 Comparison & Analytical Breakdown Pack

Section 1: Core Textbook Comparative & Analytical Exercises (11 Items)

Exercise 3.2. Communication Breakdown in Requirements Analysis

Requirements analysis is unquestionably the most communication intensive step in the software engineering process. Why does the communication path frequently break down?

Analysis of Communication Path Failures: Requirements analysis bridges two fundamentally distinct cognitive domains: domain stakeholders who understand business operations, and software engineers who understand technical architectures. Communication frequently breaks down due to five structural causes:

1. **The Semantic Vocabulary Gap:** Domain users express needs using specialized institutional terminology and acronyms that developers misinterpret, while developers explain constraints using technical abstractions (schemas, threads, protocols) that confuse users.
2. **Tacit and Unstated Knowledge:** Users operate on intuitive assumptions and routine workflows that appear so self-evident to them that they fail to mention them, leaving vital operational rules unstated.
3. **The "Yes But" Phenomenon:** Users rarely know what they truly need until they see an operational system; requirements evolve dynamically only after users begin interacting with software artifacts.
4. **Organizational and Political Conflicts:** Different stakeholder groups within the client organization often hold conflicting objectives (e.g., finance demands strict cost controls, while branch staff demand rapid flexible overrides), forcing the analyst into an unresolved mediation role.
5. **Personnel Turnover and Instability:** Client representatives frequently rotate or leave during long elicitation cycles, resulting in inconsistent instructions and contradictory revisions.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.1.1, pp. 40–42; companion Slide 134.

ملاحظة أكاديمية هامة (Academic Note)

التشخيص الهندسي: الفشل في تحليل المتطلبات ليس عيباً تقنياً، بل هو عائق اتصالي بين لغة العمل المؤسسية (*Domain Jargon*) ولغة النظم البرمجية (*Technical Abstraction*)؛ والحل يكمن في النمذجة المرئية المشتركة وورش العمل المنتظمة.

Exercise 3.9. FAST vs. Brainstorming Sessions in Requirements Elicitation

Describe Facilitated Application Specification Technique (FAST) and compare this with brainstorming sessions.

Comparative Breakdown: Both techniques are collaborative elicitation methods designed to overcome communication barriers, but they differ fundamentally in structure, governance, and output:

Dimension	Facilitated Application Specification Technique (FAST)	Brainstorming Sessions
Primary Objective	Build structured, consensual preliminary requirements and draft an initial SRS outline.	Generate a wide, unconstrained volume of creative ideas without immediate filtering.
Structural Formality	Highly structured and formal; strictly adheres to predefined agenda, preparation lists, and facilitation protocols.	Informal and dynamic; operates under open-floor, non-evaluative idea generation rules.
Facilitation Role	Managed by an objective, trained neutral facilitator who enforces rules and drives consensus.	Moderated by a leader whose primary task is simply recording ideas and preventing criticism.
Participant Preparation	Participants must prepare and bring written lists of system objects, services, and constraints.	Participants require minimal upfront preparation; ideas are sparked spontaneously.
Evaluation Timing	Ideas and objects are immediately evaluated, grouped, prioritized, and trimmed during the workshop.	Evaluation is strictly prohibited during ideation; evaluation occurs in a separate follow-up phase.
Deliverable / Output	Prioritized lists of functional requirements, system constraints, and a signed preliminary scope document.	A raw, unranked idea repository requiring subsequent analysis and filtering.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.4.2–§3.4.3, pp. 51–53; companion Slides 20–26 & Slide 135.

📝 **ملاحظة أكاديمية هامة (Academic Note)**

الفارق الجوهري في الامتحان: العصف الذهني (*Brainstorming*) يمنع النقد تماماً لجمع أكبر قدر من الأفكار الحرة، بينما ورش *FAST* تتطلب تحضيراً مسبقاً وتدمج التقييم والترتيب الفوري للوصول إلى وثيقة مواصفات مشتركة.

Exercise 3.15. Modeling Many-to-Many (M:N) Relationships as Associative Entities

Explain why a many-to-many relationship is to be modeled as an associative entity.

Architectural & Relational Rationale: An associative entity represents the association itself as an entity, which is useful when the relationship has its own attributes or needs to be represented in a relational database.

For example, ENROLLMENT can represent the association between STUDENT and COURSE and hold attributes such as grade or semester. In a relational implementation, it is commonly represented by a

linking table, replacing one M:N relationship with two 1:N relationships. The table's key design depends on the system's rules; a composite key is one possible choice.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.5.3, pp. 66–68; companion Slides 87–91 & Slide 136.

> 📝 **ملاحظة أكاديمية هامة (Academic Note)**
>
> **الفكرة:** إذا كانت للعلاقة سمات تخصّها، مثل الدرجة أو الفصل الدراسي في التسجيل، نمثل العلاقة بكيان رابط. وعند تنفيذها كجداول تتحول عادةً إلى علاقتين من نوع N:1.

Exercise 3.16. Linkages Between Data Flow Diagrams (DFD) and E-R Diagrams (ERD)

What are the linkages between data flow and E-R diagrams?

Analysis of Process-Data Duality: DFDs and ERDs show complementary views of a system: a DFD models data moving through processes, while an ERD models data entities and their relationships. They should be checked for consistency rather than treated as having a universal one-to-one mapping.

Data stores and flows in the DFD can be compared with entities, attributes, and data definitions in the ERD and data dictionary. Processes describe operations on data, but a particular store or entity need not map one-to-one to a single object in the other model.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.5.1–§3.5.3, pp. 60–68; companion Slides 82–91 & Slide 136.

> 📝 **ملاحظة أكاديمية هامة (Academic Note)**
>
> **التكامل المعماري:**
>
> - الـDFD يمثل **الأفعال والحركة** (Process Flow).
> - الـERD يمثل **الأسماء والبيانات المستقرة** (Data Persistence).
> - افحص توافق أسماء البيانات في مخازن DFD مع الكيانات والسمات المعرّفة في ERD والقاموس، من دون افتراض تطابق واحد لواحد.

Exercise 3.18. Minimum Cardinality vs. Optional and Mandatory Participation

Explain the relationship between minimum cardinality and optional and mandatory participation.

Comparative Semantic Breakdown: In data modeling, cardinality defines the numeric bounds of an association. It is expressed as a pair: $(\text{minimum cardinality}, \text{maximum cardinality})$.

- **Maximum Cardinality:** Establishes the upper limit of associated entity instances (either 1 or N).
- **Minimum Cardinality:** Establishes whether participation in the relationship is **mandatory** or **optional**.

The Exact Structural Relationship:

1. **Optional Participation ($\text{Min Cardinality} = 0$):**

- An instance of Entity A can exist in the database without being associated with any instance of Entity B.
- *Example:* EMPLOYEE assigned to PROJECT. If a newly hired employee has not yet been assigned a project, the minimum cardinality is 0. Participation of EMPLOYEE is **optional**.

2. **Mandatory Participation ($\text{Min Cardinality} \geq 1$):**
   - Every instance of Entity A must be associated with at least one instance of Entity B; existence without an associated entity violates database integrity constraints.
   - *Example:* BOOK published by PUBLISHER. A book cannot exist in the catalog without an assigned publisher. The minimum cardinality is 1. Participation of BOOK is **mandatory**.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.5.3, pp. 66–68; companion Slides 87–91 & Slide 136.

ملاحظة أكاديمية هامة (Academic Note)

قاعدة الحفظ السريعة:

- الحد الأدنى = 0 ← المشاركة اختيارية (*Optional*)؛ السجل يمكن أن يوجد وحيداً.
- الحد الأدنى ≥ 1 ← المشاركة إجبارية (*Mandatory*)؛ لا يمكن إدخال السجل دون ربطه بالكيان الآخر.

Exercise 3.26. Prototyping for Reusable Components: Benefits & Pitfalls

Discuss the significance of using prototyping for reusable components and explain the problems, which may arise in this situation.

Significance of Prototyping for Reusable Components: A prototype can help assess whether a candidate reusable component is suitable for the intended use and can clarify requirements before the final system is built.

Potential problems: Prototyping may consume more effort than expected, encourage unrealistic expectations about delivery, or lead users and developers to misunderstand the prototype's role. Tool and team experience can also affect the usefulness of the results. A prototype should therefore be planned for a clear purpose.

Problems and Pitfalls That Arise:

1. **Premature Architectural Lock-In:** Teams often develop rapid prototypes around a specific component, becoming cognitively biased toward that technology even when the prototype reveals significant architectural flaws.
2. **Superficial Feature Assumption:** A component prototype often tests basic functional workflows successfully while masking critical non-functional deficiencies (e.g., poor scalability, lack of thread safety, missing encryption).
3. **Interface Bloat and Complexity:** Integrating general-purpose reusable libraries often introduces unwanted dependencies, massive memory overheads, and bloated attack surfaces into the application.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.5.4, pp. 68–71; companion Slide 116 & Slide 138.

ملاحظة أكاديمية هامة (Academic Note)

تنبيه: نجاح prototype لغرض توضيح المتطلبات لا يثبت وحده أن المكوّن أو المنتج النهائي مناسب لكل ظروف التشغيل.

Exercise 3.27. Customer Request to Buy Prototype for Production: Developer's Response

Suppose a user is satisfied with the performance of a prototype. If he/she is interested to buy this for actual work, what should be the response of a developer?

Professional Engineering Response: First identify the prototype's purpose. If it is a throw-away prototype, explain that it was built quickly to clarify requirements and is meant to be discarded; the final system is built separately. If it is an evolutionary or validation prototype intended to become the system, assess whether it is sufficiently complete and efficient for that role before agreeing to use it operationally. A user liking its performance alone does not establish that it is ready for production.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.5.4, pp. 68–71; companion Slide 116 & Slide 138.

ملاحظة أكاديمية هامة (Academic Note)

الفكرة: رضا المستخدم عن الأداء وحده لا يكفي؛ نوع النموذج وغرضه وجودته هي التي تحدد هل يمكن أن يصبح النظام الفعلي.

Exercise 3.28. Critique: Is "Throw-Away Prototype" an Inappropriate Term?

Comment on the statement: "The term throw-away prototype is inappropriate in that these prototypes expand and enhance the knowledge base that is retained and incorporated in the final prototype; therefore they are not disposed of or thrown away at all."

The statement is partly right: the knowledge gained from a prototype is retained, but that does not mean the prototype artifact itself is retained. In a throw-away approach, the prototype is discarded after analysis and the final system is built separately. In an evolutionary approach, the prototype is intended to evolve into the final product. The term is appropriate when it describes the artifact in the first approach, not the knowledge learned from it.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.5.4, pp. 68–71; companion Slide 116 & Slide 138.

ملاحظة أكاديمية هامة (Academic Note)

التمييز: ما يُحتفظ به هو المعرفة والمتطلبات؛ أما التخلص من الشيفرة فيخصّ throw-away prototype تحديداً.

Exercise 3.29. Ambiguity Analysis of Four Requirement Statements

Which of the following statements are ambiguous? Explain why. (a) The system shall exhibit good response time. (b) The system shall be menu driven. (c) There shall exist twenty-five buttons on the control panel, numbered PF1 to PF25. (d) The software size shall not exceed 128K of RAM.

Detailed Analytical Evaluation:

1. **Statement (a): "The system shall exhibit good response time."**
   - **Status: Extremely Ambiguous.**
   - **Explanation:** The adjective **"good"** is completely subjective, qualitative, and unmeasurable. It does not define what constitutes good performance (e.g., 50 milliseconds vs. 5 seconds), under what operational workload (single user vs. 10,000 concurrent transactions), or what transaction types are being measured. It is completely untestable and unverifiable.
   - **Unambiguous Revision:** "*The system shall respond to 95% of standard database search queries in less than 1.5 seconds under a concurrent load of 500 active users.*"
2. **Statement (b): "The system shall be menu driven."**
   - **Status: Ambiguous.**
   - **Explanation:** While "menu driven" conveys a general architectural style, it lacks operational specificity. It fails to specify menu hierarchies, navigation mechanics, dynamic vs. static menus, keyboard accessibility shortcuts, or GUI widget specifications.
   - **Example revision (assumed targets):** "*The system shall display the main menu within 2 seconds after login and shall allow users to open each menu item using either the keyboard or mouse.*"
3. **Statement (c): "There shall exist twenty-five buttons on the control panel, numbered PF1 to PF25."**
   - **Status: Unambiguous.**
   - **Explanation:** This statement is mathematically precise, concrete, and directly verifiable. An inspector can physically count the buttons (exactly 25) and verify their labeling (PF1 through PF25). There is no room for subjective interpretation.
4. **Statement (d): "The software size shall not exceed 128K of RAM."**
   - **Status: Unambiguous.**
   - **Explanation:** The constraint is quantitatively bounded and measurable using standard system profiling tools. A tester can load the software and verify whether its memory footprint remains $\leq 128\ \text{KB}$.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.6.2, pp. 73–74; companion Slide 104 & Slide 138.

ملاحظة أكاديمية هامة (Academic Note)

معيار التحقق (Verifiability Rule): إذا كان بإمكانك كتابة فحص محدد بنعم/لا أو رقم كمي لقياس الشرط، فالعبارة سليمة (غير مبهمة). أما الكلمات المطاطية مثل: "جيد"، "سريع"، "سهل الاستخدام"، "حديث" فهي عبارات مبهمة ومرفوضة قطعياً في الـSRS.

Exercise 3.36 (a). Functional vs. Non-Functional Requirements

Discuss the difference between Functional and Non-Functional requirements.

Comparative Dimensional Matrix:

Dimension	Functional Requirements	Non-Functional Requirements
Core Question	*What does the system do?*	*How well does the system perform its functions?*
Definition	Statements of specific services, calculations, inputs, data processing steps, and outputs.	Constraints placed on system services and operational boundaries (performance, quality, security).
Scope of Impact	Typically localized to specific modules, transactions, or use cases.	Global and architectural; influences overall system structure, hardware selection, and data storage.
Verification Method	Verified through functional testing, black-box test cases, and use-case execution.	Verified through specialized stress testing, benchmarking, load testing, security audits, and profiling.
Failure Consequence	Specific service or transaction fails to complete properly.	Entire system may become unusable, insecure, unscalable, or non-compliant, causing total system failure.
Examples	Calculating loan interest; printing grade sheets; processing credit card transactions.	Response time ≤ 2s; 99.99% system availability; AES-256 data encryption at rest.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.2.1–§3.2.2, pp. 45–47; companion Slides 6–8 & Slide 140.

📝 **ملاحظة أكاديمية هامة (Academic Note)**

التأثير المعماري: خطأ المتطلب الوظيفي يعطل ميزة واحدة، لكن إهمال المتطلب غير الوظيفي (مثل الأمان أو التزامن) قد يفرض إعادة كتابة هيكل النظام بالكامل!

Exercise 3.36 (b). User Requirements vs. System Requirements

Discuss the difference between User Requirements and System Requirements.

Comparative Dimensional Matrix:

Dimension	User Requirements	System Requirements
Target Audience	Client executives, business managers, end-users, contract administrators.	Software architects, system designers, programmers, database engineers, testers.
Abstraction Level	High-level abstract statements of business services, operational benefits, and system goals.	Detailed, highly structured, and technical specifications of all software capabilities and constraints.

Dimension	User Requirements	System Requirements
Notation Used	Natural language prose, high-level workflow diagrams, and conceptual mock-ups.	Standardized technical language, structured templates, formal mathematical notations, DFDs, ERDs, and APIs.
Design Independence	Completely devoid of software design or architectural considerations.	Specifies precise external interfaces, data structures, and operational boundaries while avoiding internal algorithm bias.
Role in Lifecycle	Forms the primary basis for the commercial procurement contract and user acceptance criteria.	Forms the primary input for architectural design, module construction, and unit/integration verification.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.2.1–§3.2.2, pp. 45–47; companion Slides 6–8 & Slide 140.

ملاحظة أكاديمية هامة (Academic Note)

التسلسل الهرمي: متطلبات المستخدم (*User Requirements*) تُكتب بلغة العميل ليفهم ويدفع، ومتطلبات النظام (*System Requirements*) تفصلها هندسياً ليفهم المبرمج ويبني.

Section 2: Curriculum Gap-Closing Synthesis (3 High-Value Items)

Synthesis 12. Use Case Stereotypes: <<include>> vs. <<extend>>

Compare and contrast the <<include>> and <<extend>> dependency stereotypes in Use Case modeling across execution condition, arrow direction, base use case completeness, and practical examples.

Comparative Breakdown:

Dimension	<<include>> Stereotype	<<extend>> Stereotype
Execution Condition	**Unconditional and Mandatory:** The included use case executes every single time the base use case runs.	**Conditional and Optional:** The extending use case executes only if a specific extension point condition evaluates to true.
Arrow Direction	Points **from the base use case TO the included use case** (Base ⟶ Included).	Points **from the extending use case TO the base use case** (Extension ⟶ Base).
Base Completeness	The base use case is incomplete without the included use case; it cannot finish its goal without it.	The base use case is fully complete on its own; it functions normally whether the extension triggers or not.
Primary Purpose	Code/behavior reuse; refactoring common sub-routines shared across multiple use cases into one place.	Encapsulating optional, exceptional, or variant behavior without modifying or cluttering the core base flow.
Concrete Example	Withdraw Cash has an <<include>> to Verify PIN (every withdrawal unconditionally requires PIN check).	Calculate Overdraft Charge has an <<extend>> to Withdraw Cash (triggers only if account balance is negative).

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.4.5, pp. 58–59; UML 2.5 Specification.

ملاحظة أكاديمية هامة (Academic Note)

فخ السهم في الامتحان: أكبر خطأ شائع هو عكس اتجاه سهم <<extend>>! السهم في <<extend>> ينطلق دائماً من الحالة الإضافية متجهاً نحو الحالة الأصلية ليشير إلى نقطة التوسيع، بينما في <<include>> ينطلق من الأصلية نحو التابعة.

Synthesis 13. Requirements Verification vs. Requirements Validation

Compare Requirements Verification and Requirements Validation in software engineering, grounding the distinction in Barry Boehm's classic criteria.

Comparative Breakdown:

Dimension	Requirements Verification	Requirements Validation
Boehm's Criterion	*"Are we building the product right?"*	*"Are we building the right product?"*
Primary Focus	Internal consistency, structural conformance, mathematical completeness, and adherence to SRS standards.	External customer satisfaction, operational fidelity, and genuine alignment with real-world user intent.
Reference Standard	Evaluated against standard specifications, syntactic rules, and organizational templates (e.g., IEEE 830).	Evaluated against actual stakeholder operational needs, business processes, and organizational goals.
Primary Reviewers	Software quality engineers, system analysts, peer developers, and technical inspectors.	Clients, domain end-users, business managers, and external sponsors.
Techniques Used	Formal syntax checking, consistency matrix verification, DFD/ERD balancing, traceability inspections.	Prototyping demonstrations, user acceptance tests (UAT), operational walkthroughs, and field interviews.
Failure Consequence	The SRS is structurally defective, contradictory, untraceable, or unverifiable.	The software is built flawlessly according to the SRS, but solves the wrong problem and is rejected by the customer.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.7, pp. 92–94; B.B. Agarwal, S.P. Tayal & M. Gupta (2010), §4.1, p. 85 (Boehm's verification/validation questions).

ملاحظة أكاديمية هامة (Academic Note)

قاعدة بوهيم الخالدة:

- Verification ← هل كتبنا المواصفات بطريقة صحيحة هندسياً؟
- Validation ← هل كتبنا مواصفات النظام الصحيح الذي يريده المستخدم حقاً؟

Synthesis 14. Requirement Defect Classes: Inconsistent vs. Ambiguous vs. Incomplete

Analyze and contrast the three most dangerous classes of requirements defects: Inconsistent, Ambiguous, and Incomplete Requirements. Provide formal definitions, concrete examples, and primary detection methods.

Comparative Breakdown:

1. **Inconsistent Requirements (Contradictions):**
   - **Definition:** Two or more statements within the SRS assert mutually incompatible system states, functional behaviors, timing rules, or data representations.
   - **Illustrative example (created for this question):** One requirement says *"All user passwords shall expire after 30 days,"* while another says "*System administrator credentials shall never expire.*" These statements conflict if administrator credentials are included in "all user passwords."
   - **Detection Method:** Cross-referencing traceability matrices, automated decision tables, formal mathematical verification, and peer inspection sessions.
2. **Ambiguous Requirements (Multiple Interpretations):**
   - **Definition:** A requirement statement is phrased such that readers can interpret its meaning, operational bounds, or performance limits in multiple distinct ways.
   - **Concrete Example:** "*The database shall display transaction histories quickly and intuitively.*"
   - **Detection Method:** Independent party reviews (having two different developers explain how they would test the statement) and verifying whether quantifiable metrics exist.
3. **Incomplete Requirements (Omissions):**
   - **Definition:** The SRS fails to specify necessary system responses to specific input combinations, boundary conditions, hardware failures, or exceptional states.
   - **Concrete Example:** The SRS defines what happens when a student enters valid marks (1 to 100), but completely omits what the software should do when a user enters negative marks, letters, or leaves the field blank.
   - **Detection Method:** Boundary value analysis, state-transition matrix completeness checks, and structured review checklists targeting error handling and edge cases.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.6.2, pp. 73–75; IEEE Std 830-1998.

ملاحظة أكاديمية هامة (Academic Note)

خلاصة عيوب المتطلبات:

- Inconsistent: معلومتان متناقضتان تصطدمان ببعضهما.
- Ambiguous: معلومة واحدة تحتمل أكثر من تفسير.
- Incomplete: معلومة مفقودة تماماً سكت عنها المحلل.

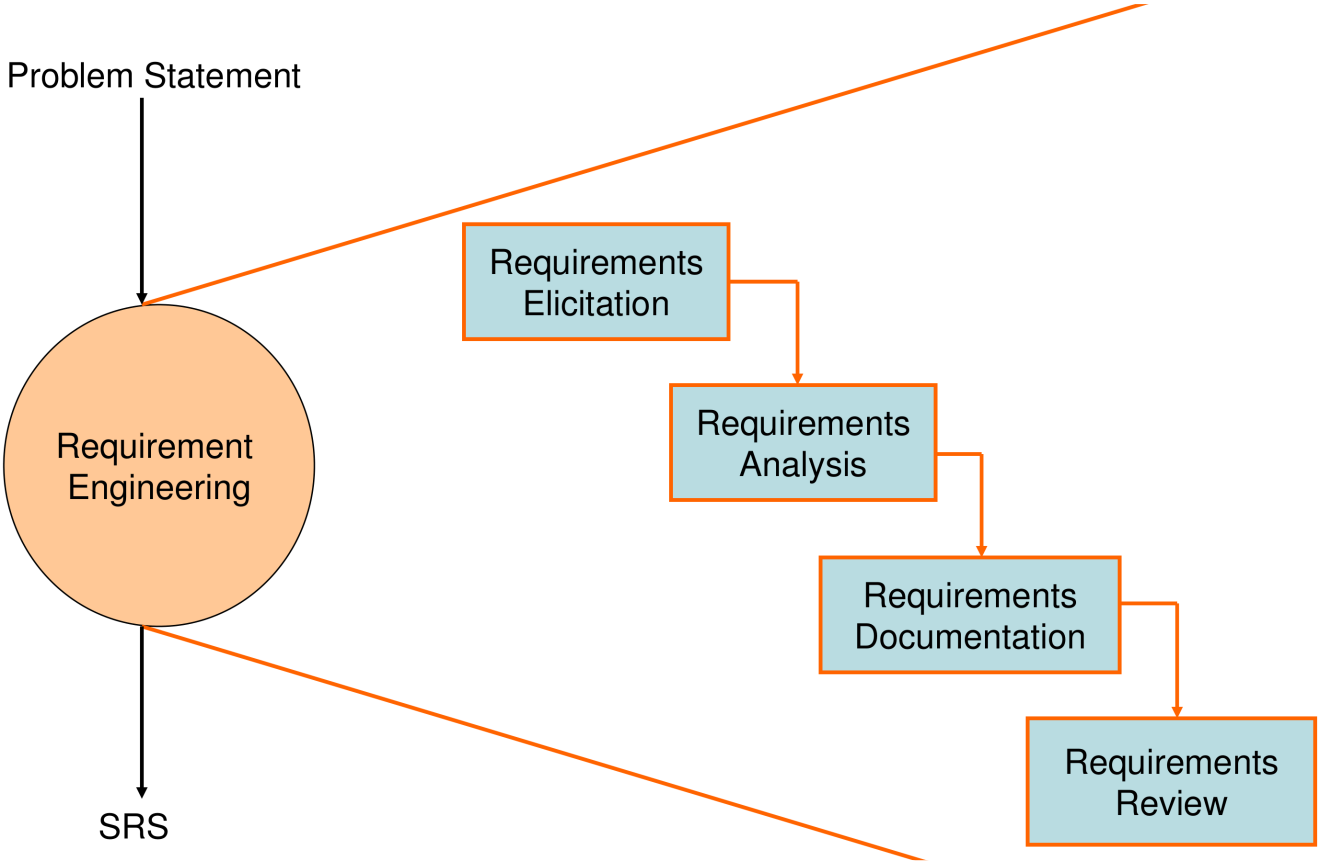
ASE Chapter 3

T4 Architectural Diagrams & Visual Modeling Pack

Section 1: Core Architectural & Process Models (5 Model Units)

Diagram 01. The Four Core Requirements Engineering Activities (Figure 3.1)

Name the input, the four core requirements-engineering activities, and the output shown in the figure.



Requirements engineering process figure

Process shown in the figure:

Input: Problem Statement.

Activities: Requirements Elicitation → Requirements Analysis → Requirements Documentation → Requirements Review.

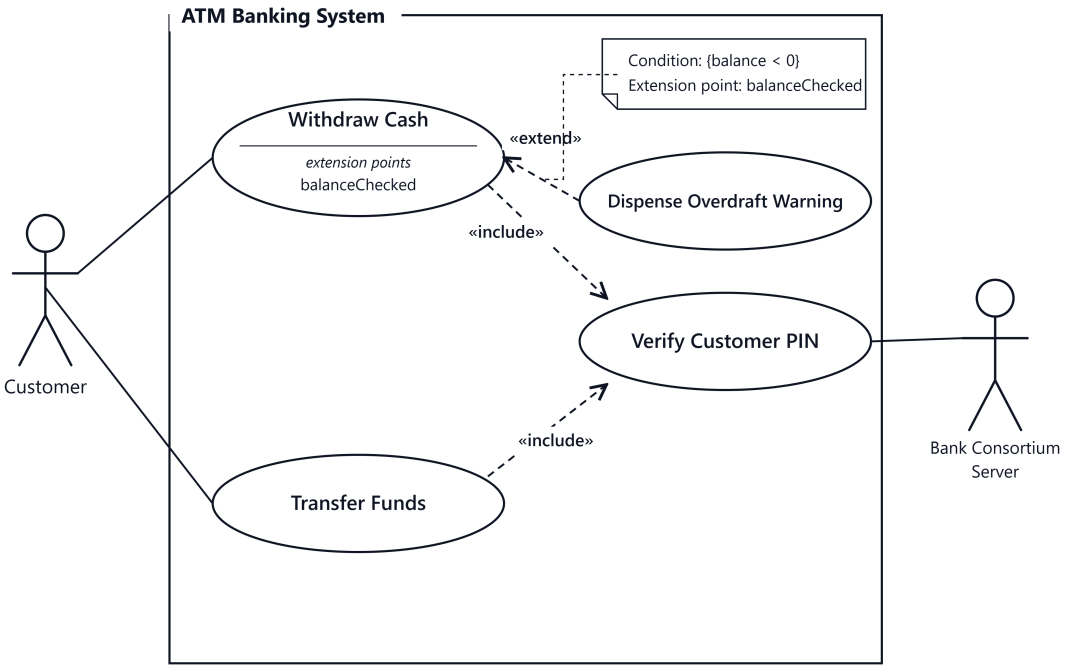
Output: SRS.

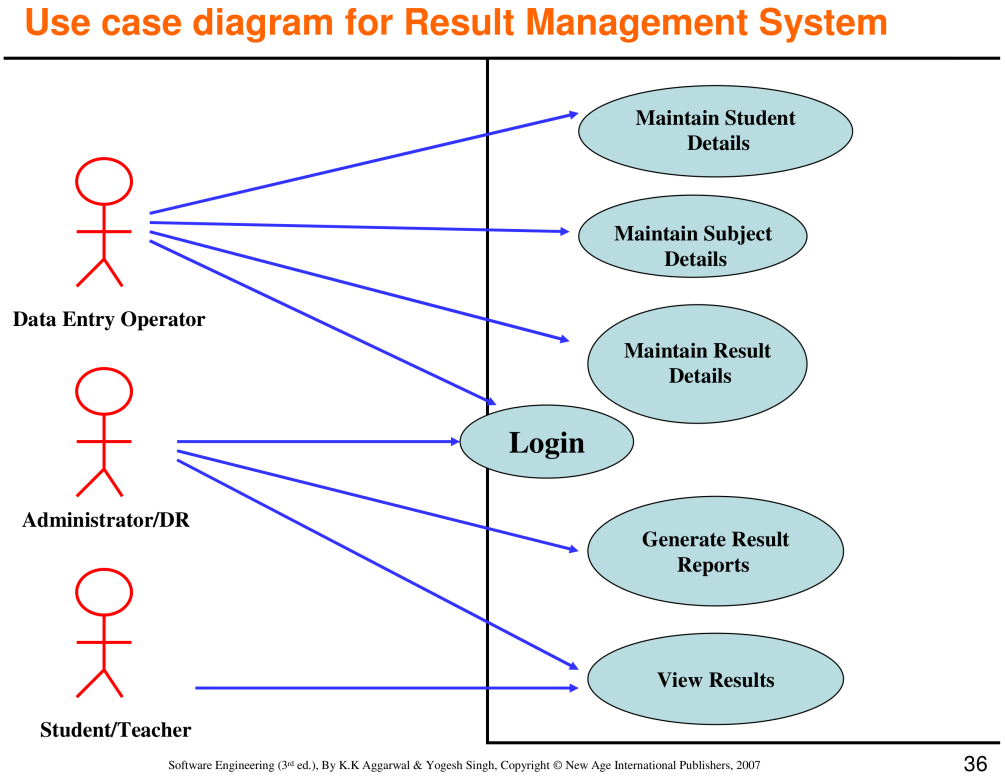
Source: companion Slide 3; K.K. Aggarwal & Yogesh Singh (3rd ed.), Fig. 3.1, p. 41.

> **ملاحظة أكاديمية هامة (Academic Note)**
> الشكل يعرض ترتيب الأنشطة الأربع من دون أسهم رجوع؛ لا يوضّح وحده حلقات مراجعة أو قرار جدوى.

Diagram 02. Use Case Model Architecture: Boundary, Actors, & Stereotypes

Diagram and specify the structural architecture of a UML Use Case Model, illustrating system boundary, external actors, primary use cases, and the defining distinctions between <<include>> and <<extend>> relationships.





Structural & Behavioral Breakdown:

1. **System Boundary:** The rectangular boundary encloses all automated software capabilities, explicitly separating internal functions from external entities.
2. **Primary and Secondary Actors:**
   - Customer (Primary): Initiates interactions to achieve a business goal (Withdraw Cash, Transfer Funds).
   - Bank Consortium Server (Secondary): External backend infrastructure that authenticates and processes transactions.
3. **The <<include>> Stereotype (Mandatory Dependency):**
   - Unconditionally invoked whenever the base use case executes. Both Withdraw Cash and Transfer Funds unconditionally include Verify Customer PIN. The arrow points **from Base Use Case to Included Use Case**.
4. **The <<extend>> Stereotype (Conditional Extension):**
   - Executed only if a specific condition is met at an extension point. Dispense Overdraft Warning extends Withdraw Cash only when the balance drops below zero. The arrow points **from Extending Use Case to Base Use Case**.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.3.4, pp. 58–61; companion Slide 36 for the system example; OMG UML 2.5.1 UseCases clause for include/extend semantics.

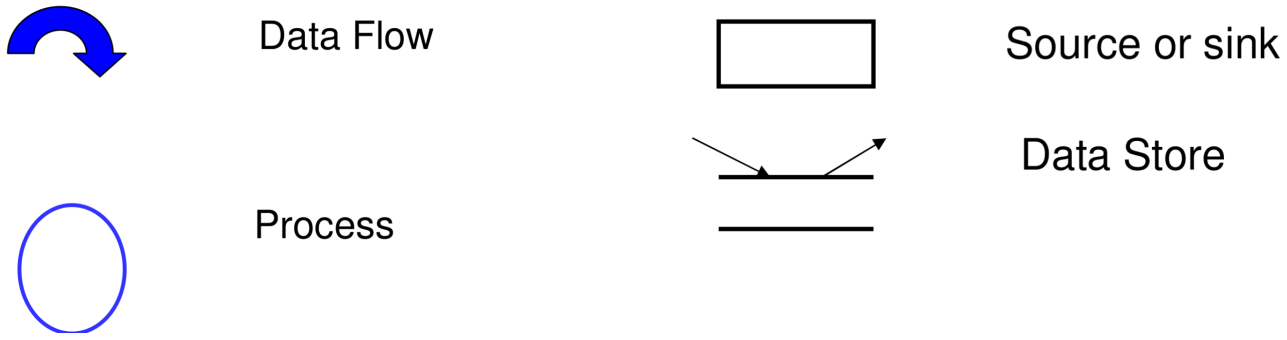
ملاحظة أكاديمية هامة (Academic Note)

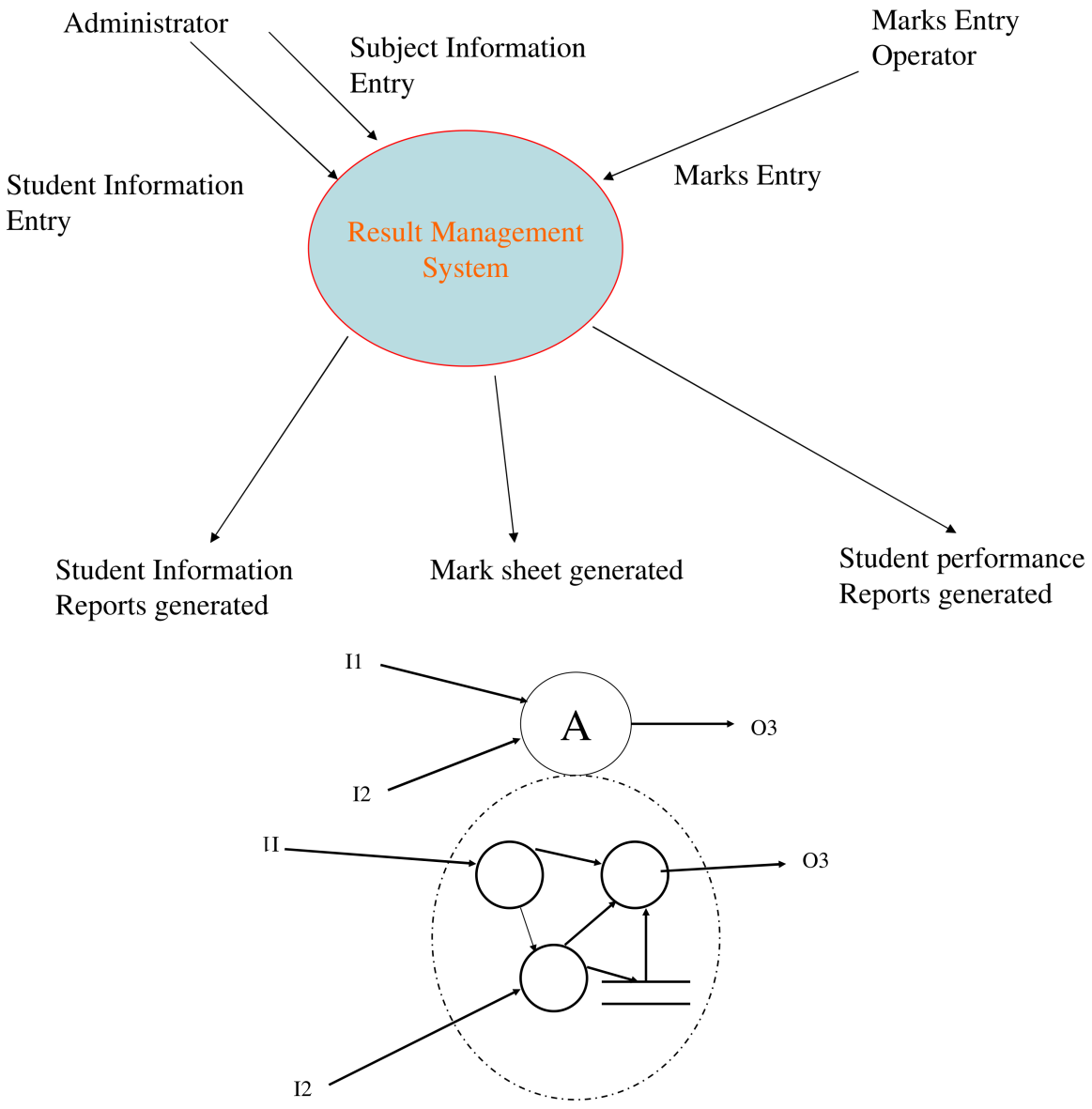
قاعدة السهم في الامتحان:

- في <<include>>: السهم ينطلق من الحالة الأصلية نحو الحالة التابعة (Withdraw Cash → Verify PIN).
- في <<extend>>: السهم ينطلق من الحالة التوسعية الاختيارية عائداً إلى الحالة الأصلية (Overdraft Warning → Withdraw Cash).

Diagram 03. Data Flow Diagram (DFD): Symbols and Separate Examples

Name the four DFD symbols shown, then identify what the publisher examples on Slides 83 and 86 each represent.





DFD decomposition and boundary-flow example

The four symbols:

- **Process:** circle or bubble.
- **Data flow:** directed arrow.
- **Data store:** open or parallel-line symbol.
- **External entity:** rectangle.

Slide 83 shows a Results Management context diagram. Slide 86 shows a separate generic context-to-levelled example using `I1`, `I2`, and `O3`. They are different examples, not a parent diagram and its balanced decomposition. In a genuine decomposition, the child diagram must preserve the parent process's external input and output flows.

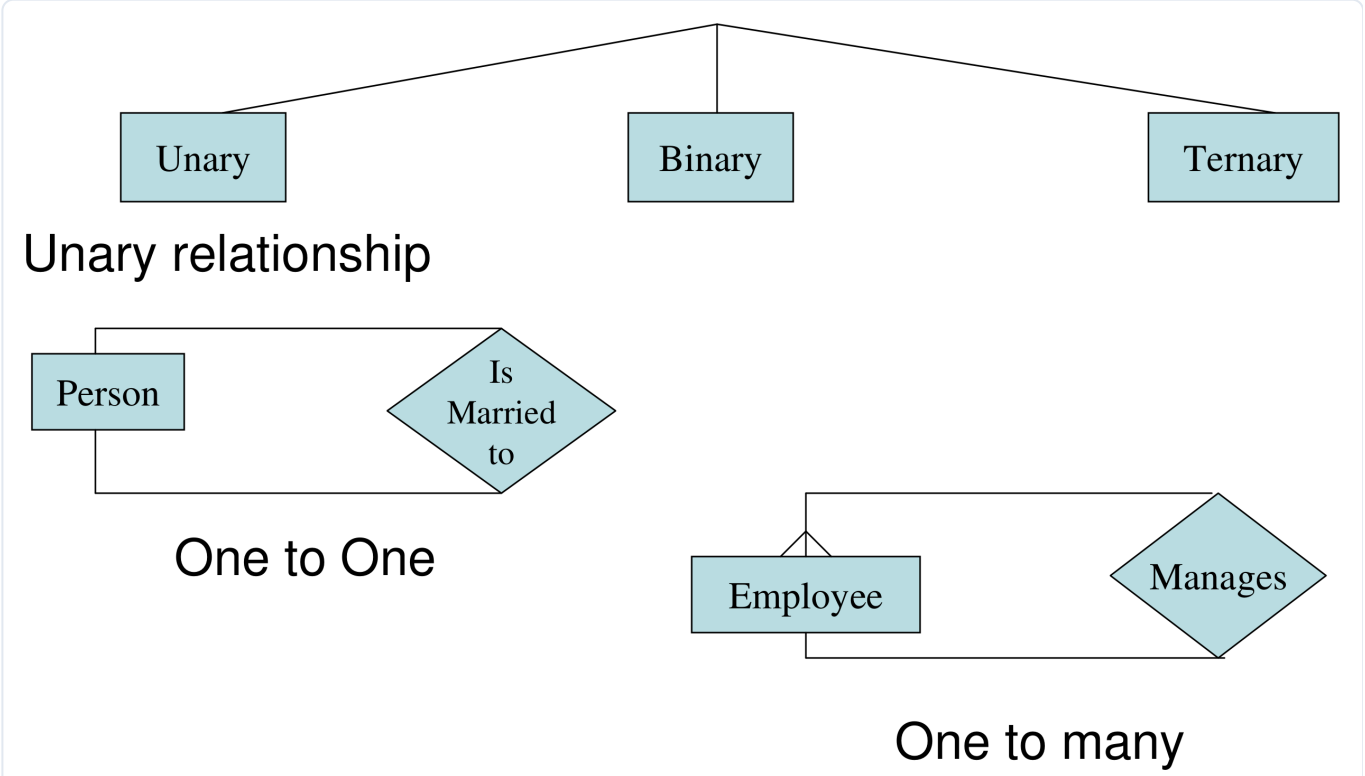
Source: companion Slides 83–86; K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.5.1, pp. 61–65.

ملاحظة أكاديمية هامة (Academic Note)

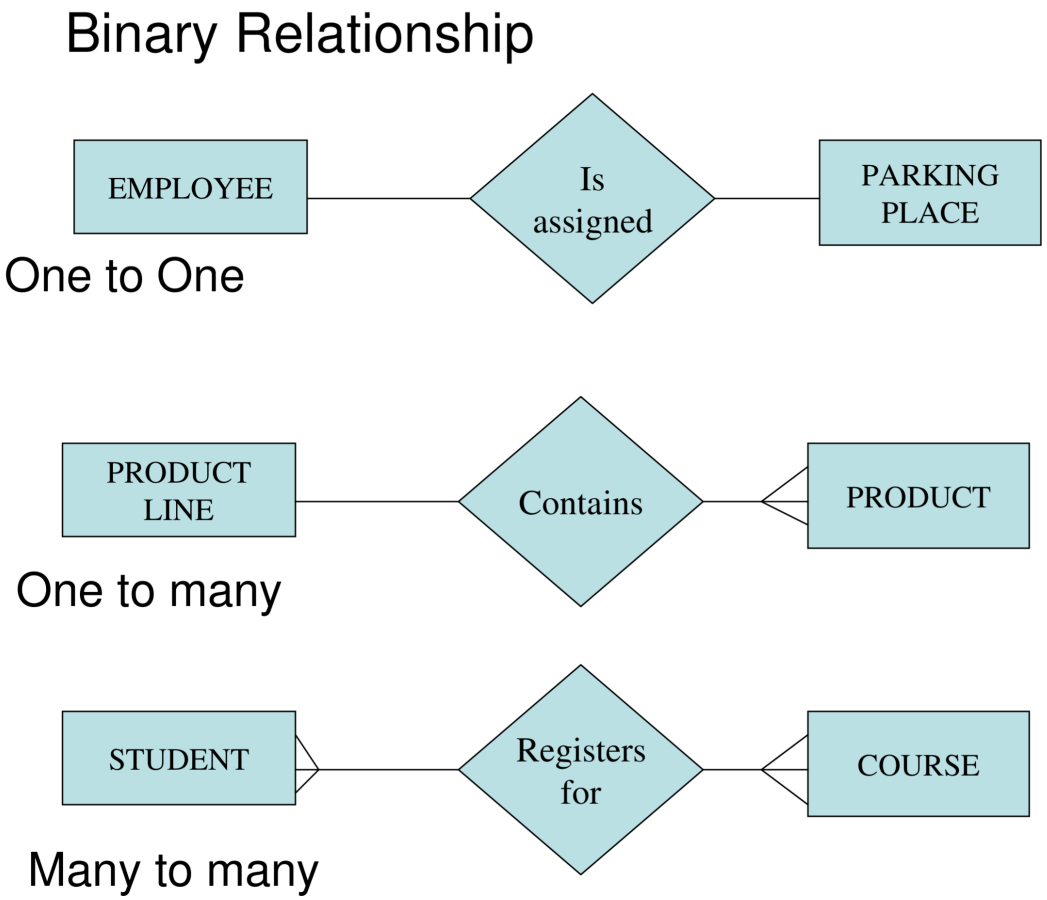
لا نخلط بين مخطط السياق وDFD المستوى الأول في الشريحة 86 وبين مخطط النتائج المستقل في الشريحة 83؛ الشريحتان لا تعرضان زوجاً متوازناً واحداً.

Diagram 04. Entity-Relationship (ER) Architecture & Associative Entity Decomposition

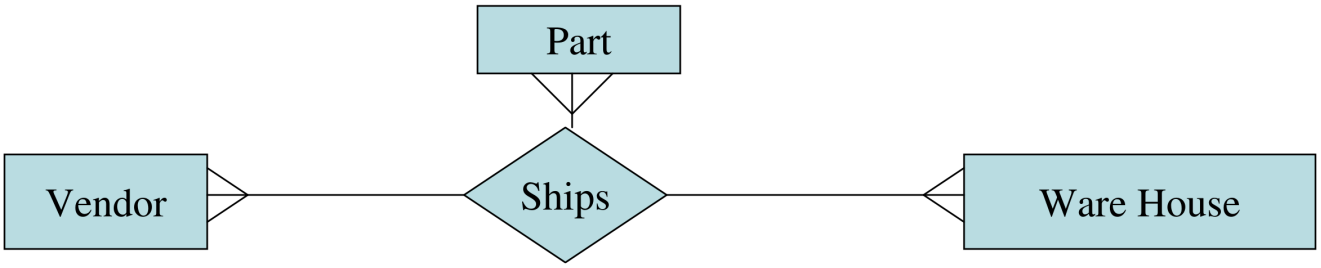
Diagram the core components of an Entity-Relationship Diagram (ERD), illustrate relationship degrees (Unary, Binary, Ternary), and show one relational mapping option for a Many-to-Many (M:N) relationship.



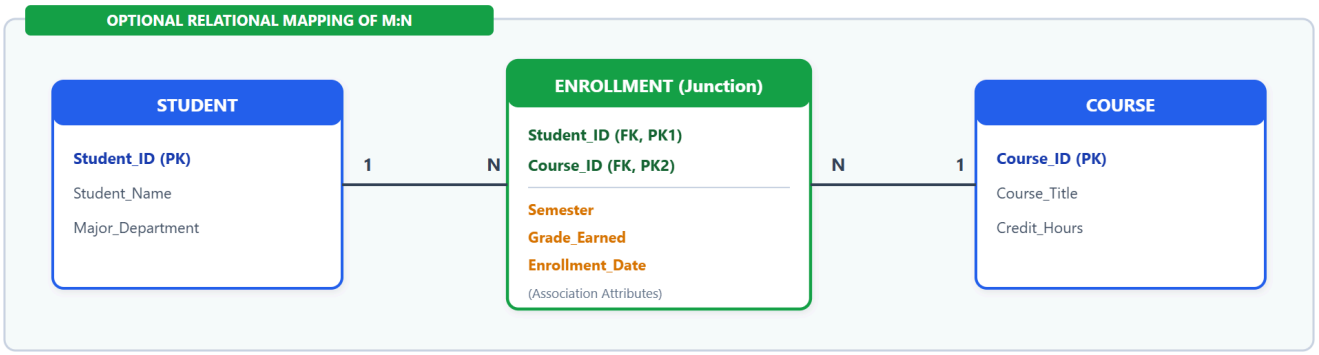
ERD relationship degrees and unary examples



Binary ERD relationship examples



Ternary ERD relationship example



Many-to-many relational mapping illustration

Structural & Relational Walkthrough:

1. **Core ERD Constructs:**
   - **Entity (Rectangle):** Real-world object having independent existence (STUDENT, COURSE).
   - **Attribute (Ellipse):** Properties describing an entity (Student_Name, Credit_Hours).
   - **Relationship (Diamond):** Association between entity instances.
2. **Relationship Degrees:**
   - **Unary (Degree 1):** Associating instances of the same entity type (an EMPLOYEE manages other EMPLOYEEs).
   - **Binary (Degree 2):** Associating instances of two distinct entity types (STUDENT and COURSE).
   - **Ternary (Degree 3):** Simultaneous association among three entity types (DOCTOR, PATIENT, MEDICATION).
3. **A Relational Implementation of M:N:**
   - The conceptual ER model can show a direct Many-to-Many relationship, including attributes of that relationship.
   - When mapping that relationship to relational tables, one common implementation is an **Associative Entity / Junction Table (ENROLLMENT)** with foreign keys such as (Student_ID, Course_ID) and relationship attributes such as Grade, Semester, or Registration_Date. This is a relational mapping option, not a mandatory ER-diagram rule.

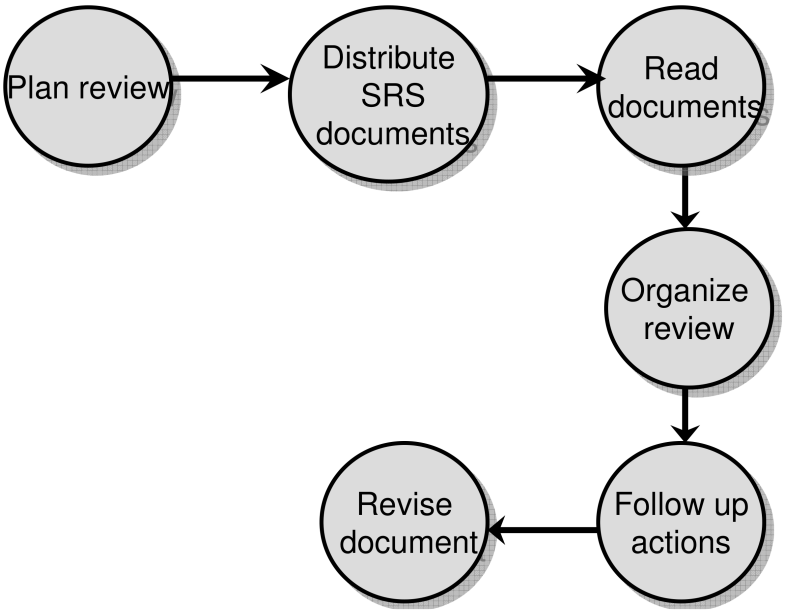
Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.5.2, pp. 66–68; companion Slides 89–95.

ملاحظة أكاديمية هامة (Academic Note)

عند التحويل إلى جداول علائقية: يمكن تمثيل علاقة (M:N) بأحد الخيارات باستخدام كيان رابط (*Associative Entity*) يحمل مفاتيح الكيانين وخصائص العلاقة، مثل الدرجة أو تاريخ التسجيل.

Diagram 05. Requirements Review & Validation Process Workflow (Figure 3.21)

List the six stages of the Requirements Review Process in the order shown.



Six-stage requirements review process

Six stages:

1. Plan review.
2. Distribute SRS documents.
3. Read documents.
4. Organize review.
5. Follow up actions.
6. Revise document.

Source: companion Slide 113; K.K. Aggarwal & Yogesh Singh (3rd ed.), Fig. 3.21, pp. 92–94.

ملاحظة أكاديمية هامة (Academic Note)

الشرائح 114–115 تعرض إجراءات معالجة المشكلات وقائمة التحقق على نحو منفصل؛ الشكل هنا يعرض تسلسل المراحل الست فقط.

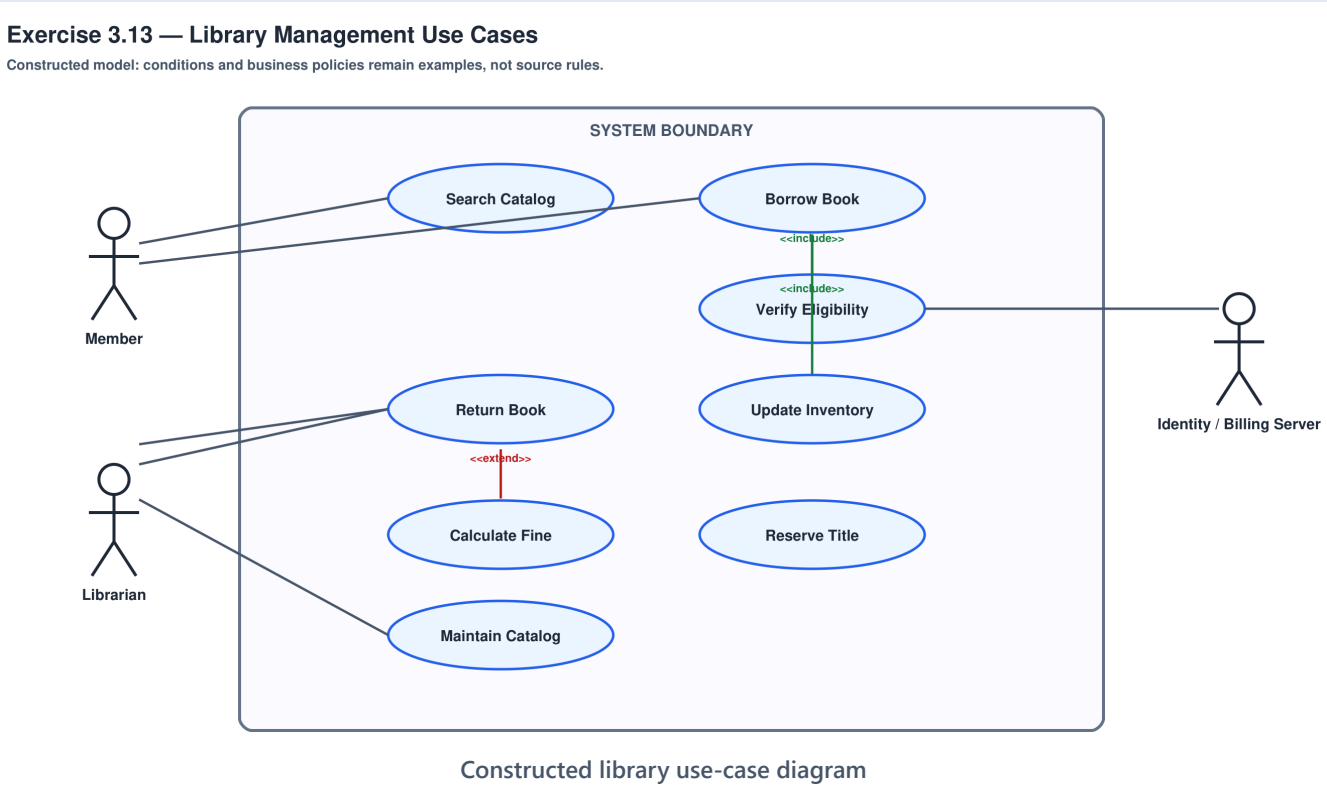
ASE Chapter 3

T5 Applied Engineering & System Creation Pack

Section 1: Core Applied Textbook Systems & Scenarios (8 Items)

Exercise 3.13. Library Management System: Problem Statement, Boundary, & Use Cases

For a Library Management System, design (i) a problem statement, (ii) its use-case diagram architecture, and (iii) formal structured use cases.



Constructed library use-case diagram

(i) Problem Statement: A university library needs a system to manage its catalog, member borrowing, reservations, returns, and overdue fines. Members must be able to search for books and borrow eligible available copies; library staff must maintain catalog records and review circulation activity.

(ii) Use Case Architecture & Boundary:

- **System Boundary:** Automated Library Information System （ALIS）.
- **Primary Actors:** Member (Student / Faculty), Librarian.
- **Secondary External Actor:** University Billing / Identity Server.
- **Core Use Cases:**
  - Search Catalog (Public access)
  - Borrow Book (Includes Verify Member Eligibility and Update Inventory)

    - Calculate Overdue Fine (extends Return Book when the book is overdue)
    - Reserve Title (Executed when zero available copies exist)
    - Catalog New Accession (Restricted to Librarian)

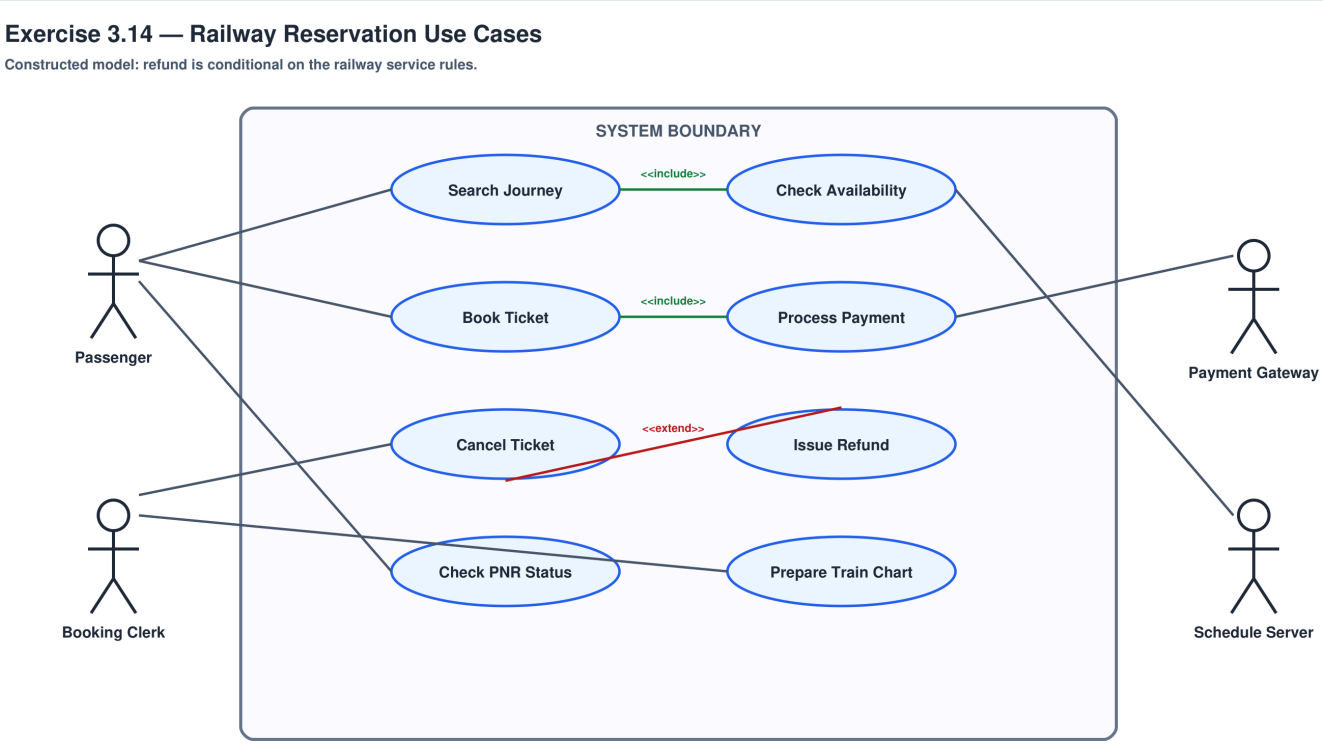
(iii) Formal Structured Use Case (Borrow Book):

- **Use Case Name:** Borrow Book
- **Primary Actor:** Member (facilitated by Librarian or Self-Checkout Kiosk)
- **Preconditions:** The member is identified and eligible to borrow under the library's rules; a copy is available.
- **Main Success Scenario (Happy Path):**
    1. The member requests an available book.
    2. The system checks the member's eligibility and confirms that a copy is available.
    3. The system records the loan and sets a due date according to library policy.
    4. The system updates the copy's status and confirms the loan to the member.
- **Alternative & Exception Flows:**
    - *Copy unavailable:* The system offers the member the option to reserve the book.
    - *Borrowing limit reached:* The system rejects the loan and explains the restriction.
    - *Account blocked by unpaid fines or another library rule:* The system explains what must be resolved before borrowing.

Prompt: K.K. Aggarwal & Yogesh Singh (3rd ed.), companion Slide 135. Use-case structure: §3.3.4, pp. 58–61. Worked design: constructed example.

Exercise 3.14. Railway Reservation System: Problem Statement, Actors, & Use Cases

Consider the problem of a Railway Reservation System and design the following: (i) Problem Statement (ii) Use Case Diagram Architecture (iii) Formal Structured Use Cases



Constructed railway-reservation use-case diagram

(i) Problem Statement: A railway agency needs an online system to manage train schedules, seat availability, passenger bookings, cancellations, and booking status. Passengers should be able to search for journeys, book available seats, and receive confirmation; staff should maintain schedule and service information.

(ii) Use Case Architecture & System Boundary:

- **System Boundary:** Railway Reservation & Ticketing Engine.
- **Primary Actors:** Passenger, Booking Clerk.
- **Secondary External Actors:** Payment Gateway, Central Train Scheduling Server.
- **Core Use Cases:**
  - Search Journey (includes Check Availability)
  - Book Journey Ticket (includes Process Payment)
  - Cancel Ticket
  - Issue Cancellation Refund (extends Cancel Ticket when refund rules allow it)
  - Check PNR Status
  - Prepare Train Chart (scheduled staff/system task)

(iii) Formal Structured Use Case (Book Journey Ticket):

- **Use Case Name:** Book Journey Ticket
- **Primary Actor:** Passenger
- **Preconditions:** The passenger has selected a route and travel date and supplied the required booking details.
- **Main Success Scenario:**
    1. The passenger submits the journey and passenger details.
    2. The system checks seat availability and temporarily holds an available seat while payment is processed.
    3. The system invokes <<include>> Process Payment through the payment service.
    4. If payment succeeds, the system confirms the booking, records it, and sends the passenger a ticket and booking reference.
- **Extensions & Exceptions:**
    - *No seat is available:* The system reports that the requested journey cannot be booked; it may offer a waitlist if the service supports one.
    - *Payment fails:* The system cancels the pending booking, releases the temporary seat hold, and informs the passenger.

Prompt: K.K. Aggarwal & Yogesh Singh (3rd ed.), companion Slide 136. Use-case structure: §3.3.4, pp. 58–61. Worked design: constructed example.

ملاحظة أكاديمية هامة (Academic Note)

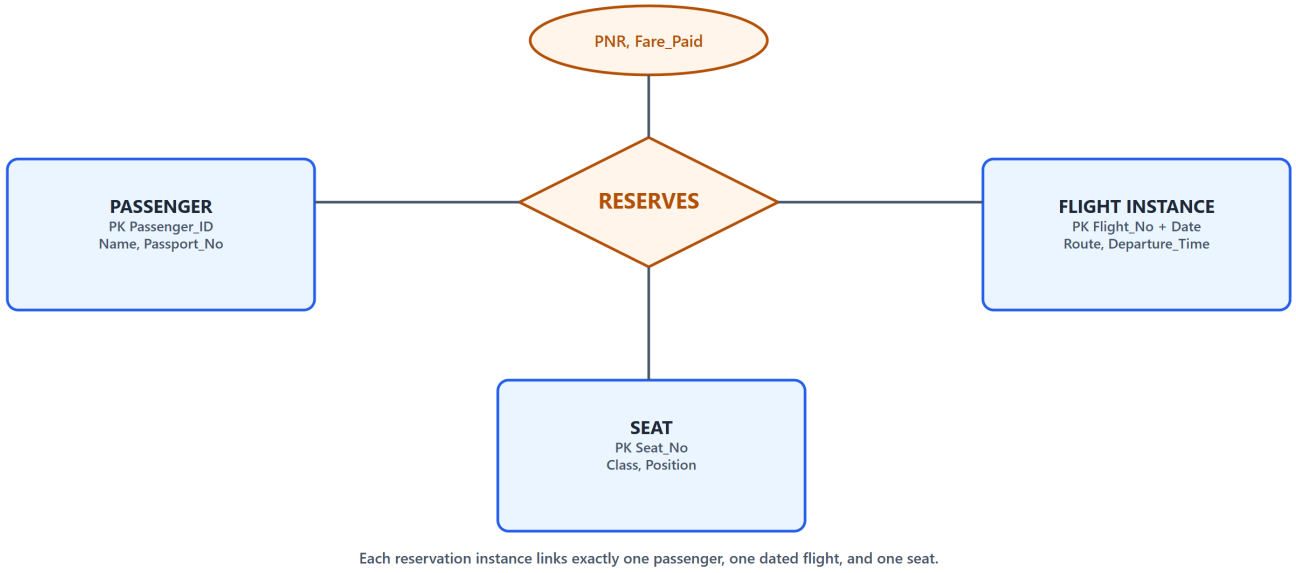
قاعدة حجز المقاعد البرمجية: حجز التذاكر في الأنظمة المتزامنة يعتمد على قفل مؤقت للمقعد (*Temporary Lock*) أثناء الدفع؛ فإذا فشلت بوابة الدفع يعود المقعد للمخزون فوراً لتجنب المقاعد المعلقة.

Exercise 3.19. Airline Reservation E-R Modeling: Passenger, Flight, & Seat Associations

An airline reservation is an association between a passenger, a flight, and a seat. Select a few pertinent attributes for each of these entity types and represent a reservation in an E-R diagram model.

Exercise 3.19 — Airline Reservation as a Ternary ER Relationship

The source describes an association among passenger, flight, and seat; the date identifies a flight instance.



Each reservation instance links exactly one passenger, one dated flight, and one seat.

Constructed airline reservation ER diagram

Ternary E-R Model:

- PASSENGER (**Passenger_ID**, Full_Name, Passport_Number, Phone_Number)
- FLIGHT INSTANCE (**Flight_Number, Flight_Date**, Origin, Destination, Departure_Time)
- SEAT (**Seat_Number**, Class, Position)
- A ternary RESERVATION relationship connects one passenger, one dated flight instance, and one seat. Relationship attributes may include PNR, Booking_Time, and Fare_Paid.

Cardinality and identity assumptions:

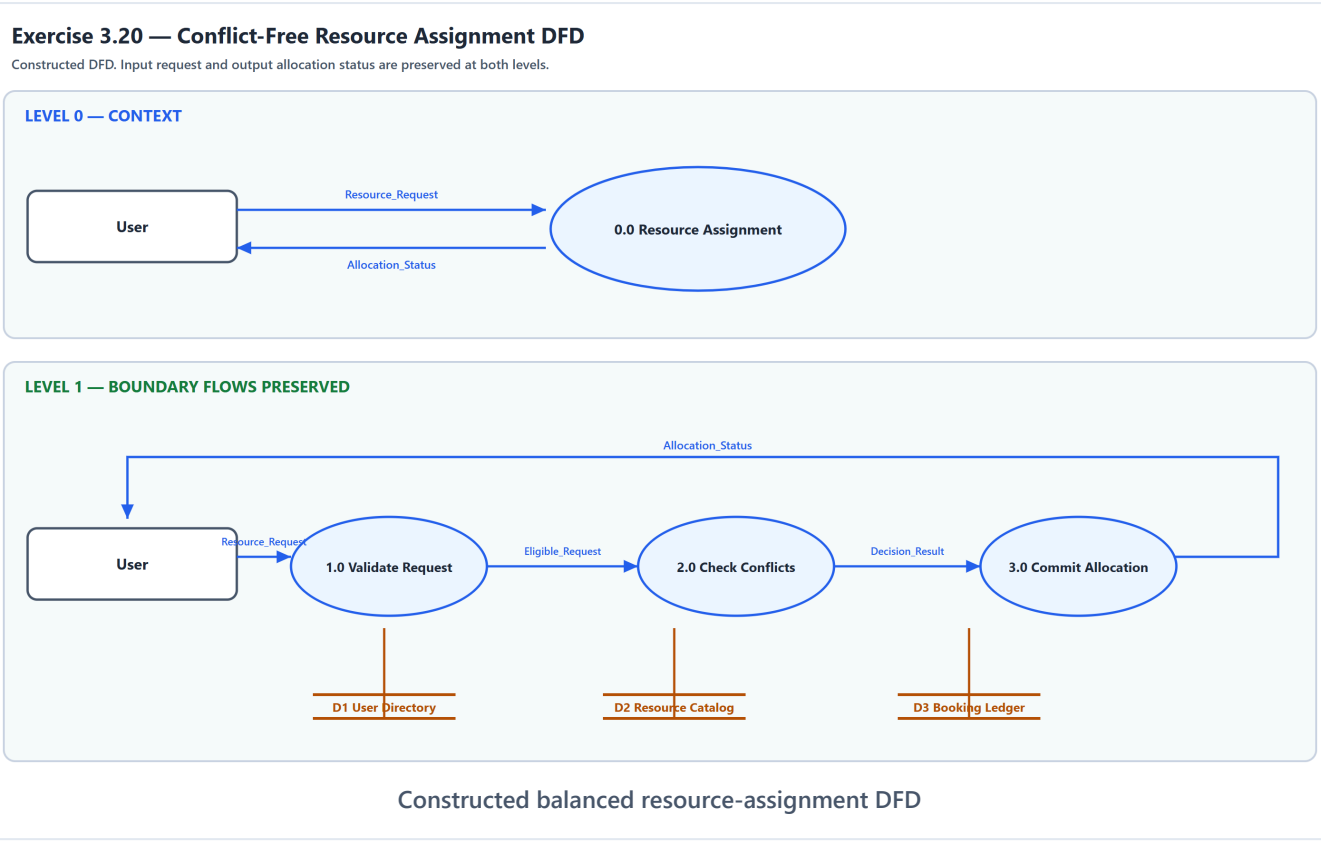
- One reservation instance refers to exactly one passenger, one flight instance, and one seat.
- A passenger may make zero or many reservations; a dated flight may have zero or many reservations; a seat may be associated with many reservations across different flight dates.
- To prevent double booking, assume a seat is assigned to at most one active reservation for a given dated flight. This is a business constraint on (Flight_Number, Flight_Date, Seat_Number), not a general ER cardinality rule.
- Treat the date as part of flight-instance identity because a flight number can operate on multiple dates.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.5.2, pp. 66–68; companion Slide 136.

> **ملاحظة أكاديمية هامة (Academic Note)**
> **قيد تشغيلي مفترض:** المقعد الواحد لا يُحجز مرتين على الرحلة ذات التاريخ المحدد. هذا قيد على بيانات الحجز، وليس قاعدة عامة لتعددية ER.

Exercise 3.20. Conflict-Free Computer Resource Assignment System DFD

A department of computer science has usual resources and usual users for these resources. A software is to be developed so that resources are assigned without conflict. Draw a DFD specifying the above system.



Constructed balanced resource-assignment DFD

Functional Architecture & DFD Decomposition:

Level-0 Context Diagram:

- **External Entity:** User (faculty member, researcher, or student); an administrator may act through the same system interface.
- **Central Process:** `0.0` Resource Assignment & Scheduling System.
- **Input Flow:** Resource_Request (User_ID, Resource_Type, Date, Time_Slot).
- **Output Flow:** Allocation_Status (confirmation or rejection).

Level-1 Detailed Functional Processes:

1. **Process 1.0: Validate User Eligibility & Request Format:**
    - Accepts Resource_Request; checks D1 User Directory to verify active academic standing and resource quota privileges.

2. **Process 2.0: Conflict Detection & Schedule Matching:**
    - Queries D2 Resource Master Catalog (listing Labs, Servers, Projectors, GPUs) and D3 Schedule Booking Ledger.
    - Evaluates whether requested Resource_ID is free for (Date, Time_Slot).
    - *If available:* Computes assignment and passes to allocation process.
    - *If conflict exists:* Formulates alternative available slots and triggers rejection/rescheduling notification.
3. **Process 3.0: Commit Allocation & Return Status:**
    - Records an approved allocation in D3 Schedule Booking Ledger; returns Allocation_Status to the user whether the request is accepted or rejected.

Balancing & Conservation: The context and Level-1 diagrams use the same external flows: Resource_Request enters from User, and Allocation_Status returns to User.

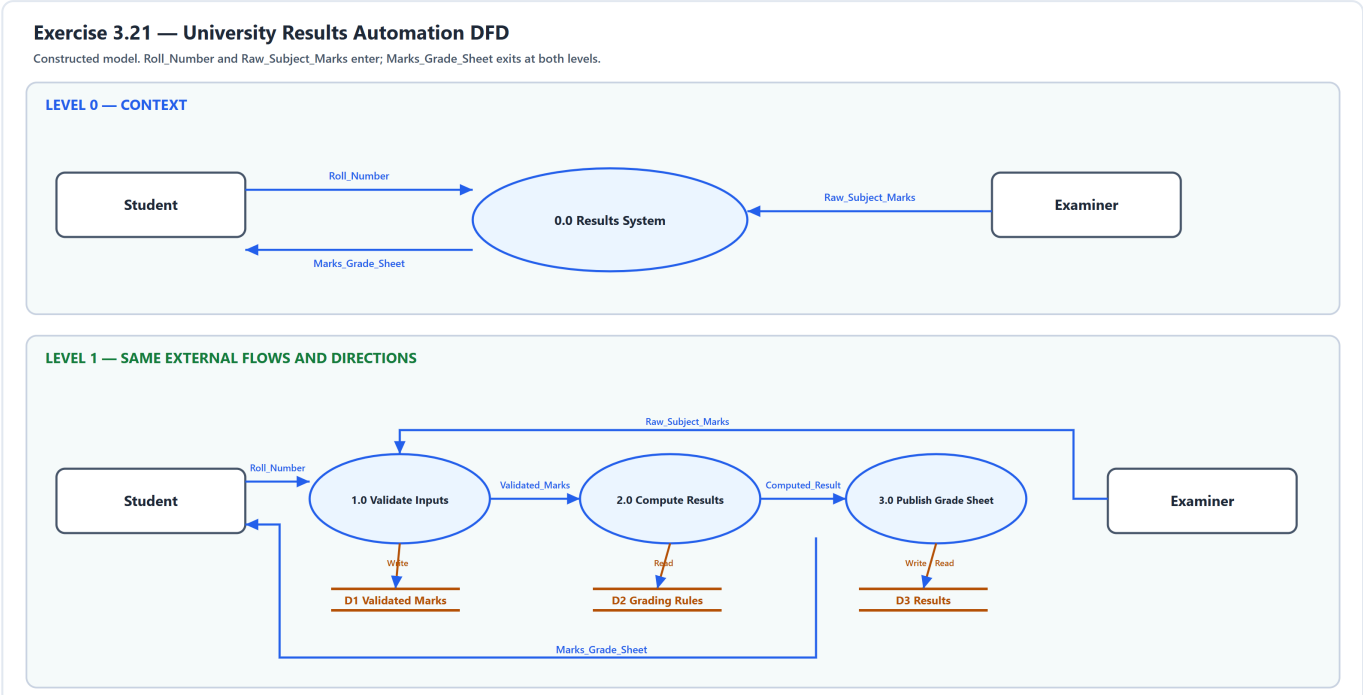
Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.5.1, pp. 61–65; companion Slide 137.

ملاحظة أكاديمية هامة (Academic Note)

خوارزمية منع التضارب (Conflict-Free Guarantee): قلب النظام يكمن في العملية 2.0؛ حيث يتم فحص جدول الحجوزات (D3 Schedule Ledger) بقفل متزامن للتأكد من عدم وجود حجز متداخل في نفس الفترة الزمنية قبل كتابة التأكيد.

Exercise 3.21. University Result Preparation Automation System DFD

Draw a DFD for result preparation automation system of B. Tech. courses (or MCA program) of any university. Clearly describe the working of the system. Also mention all assumptions made by you.



Constructed university-results DFD

Operational Assumptions:

1. Each semester offers a fixed curriculum of core and elective courses; for this constructed example, each subject has 40 internal and 60 external marks (100 total).
2. Minimum passing threshold is 40% in individual components and 50% aggregate.
3. Letter grades and Grade Point Averages (GPA) are calculated using standardized university conversion tables.

System Working & Functional Flow:

1. **Marks Ingestion:** Faculty or examination evaluators enter raw internal and external assessment marks via roll numbers.
2. **Verification & Audit:** Marks are checked against maximum bounds (0–40, 0–60); anomalous entries are rejected.
3. **Result Calculation:** The engine computes total marks, maps scores to Grade Points, evaluates Pass/Fail/Backlog status, and calculates Semester Grade Point Average (SGPA).
4. **Tabulation & Publishing:** Generates an individual grade sheet for the student from the verified marks and calculated result.

Level-1 DFD Process Breakdown:

- **Process 1.0 (Validate Inputs):** Accepts Roll_Number from the student and Raw_Subject_Marks from the examiner; validates and stores the marks.
- **Process 2.0 (Compute Results):** Applies the assumed grading rules to validated marks and stores the computed result.
- **Process 3.0 (Generate Student Grade Sheet):** Formats and returns Marks_Grade_Sheet to the student using the computed result.

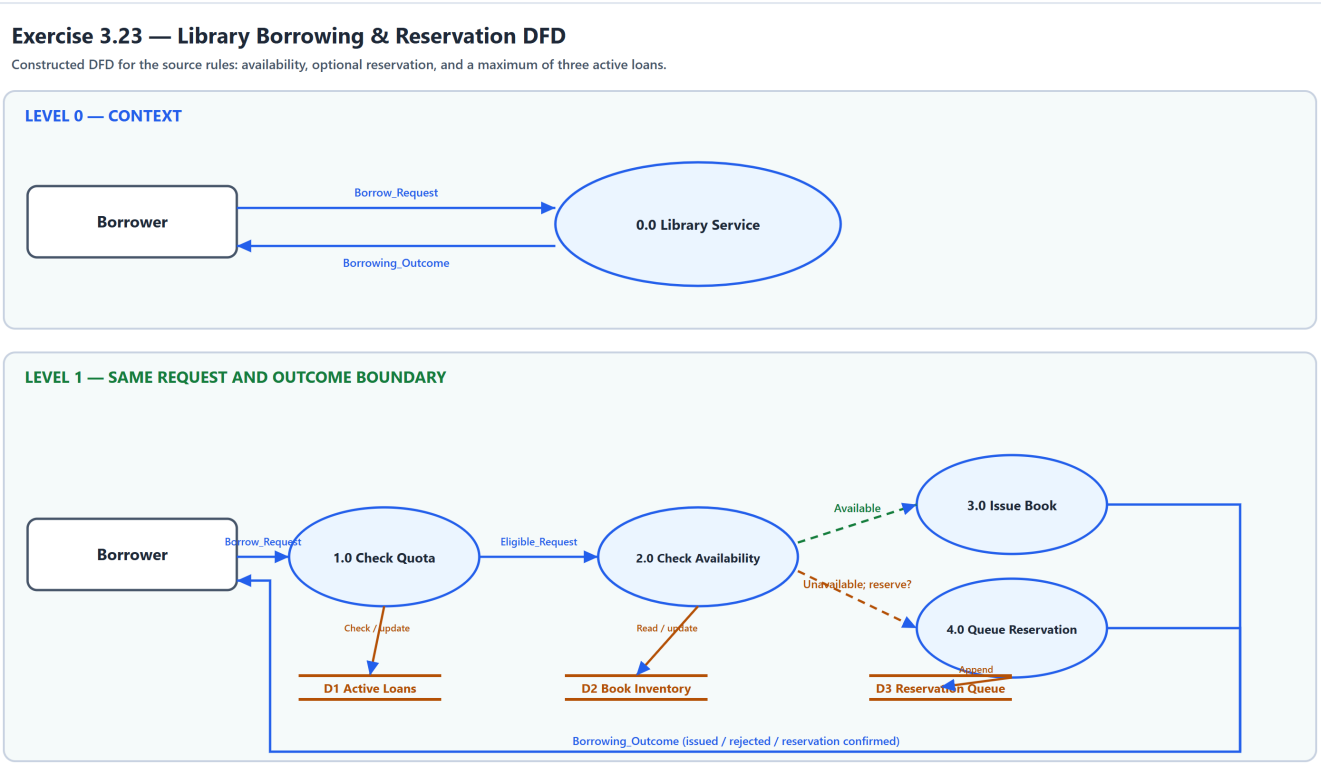
Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.5.1, pp. 61–65; companion Slide 137.

ملاحظة أكاديمية هامة (Academic Note)

حدود المثال: تقسيم 40/60 يظهر في نموذج الـSRS بالكتاب (ص. 124–125). عتبات النجاح وقواعد احتساب المعدل في هذا الحل افتراضات توضيحية؛ لا ينص عليها مثال الكتاب المذكور.

Exercise 3.23. Library Book Borrowing & Reservation DFD with Business Constraints

Draw a DFD for borrowing a book in a library which is explained below: "A borrower can borrow a book if it is available else he/she can reserve for the book if he/she so wishes. He/she can borrow a maximum of three books."



Constructed library borrowing DFD

Functional Specification & Constraint Modeling:

Level-1 DFD Processes & Data Stores:

1. **Process 1.0: Check Borrower Eligibility & Quota Limit:**
   - Input: Borrower_ID from Borrower.
   - Reads: D1 Borrower Master File and counts current borrowed books in D2 Circulation Active Loans.
   - **Constraint Check:** If $\text{Count} \geq 3$, dispatch Quota_Exceeded_Notice and terminate flow. If $\text{Count} < 3$, forward Borrower_ID to Process 2.0.
2. **Process 2.0: Check Book Inventory Availability:**
   - Input: Book_Accession_Number from Borrower.
   - Reads: D3 Book Master Inventory.
   - **Branching Decision:**
     - *Path A (Available):* Route to Process 3.0 (Issue Book).
     - *Path B (Unavailable / Already Borrowed):* Prompt borrower with Book_Not_Available_Notice and offer option to route to Process 4.0 (Process Reservation).
3. **Process 3.0: Issue Book & Update Records:**

- Updates book status in D3 Book Master Inventory to Borrowed.
- Inserts new loan record in D2 Circulation Active Loans with due date.
- Outputs: Issued_Book and Issue_Slip to Borrower.

4. **Process 4.0: Queue Title Reservation:**
   - If borrower accepts reservation offer:
   - Appends (Borrower_ID, Book_ID, Timestamp) to D4 Reservation Queue.
   - Outputs: Reservation_Confirmation_Slip indicating current position in queue.

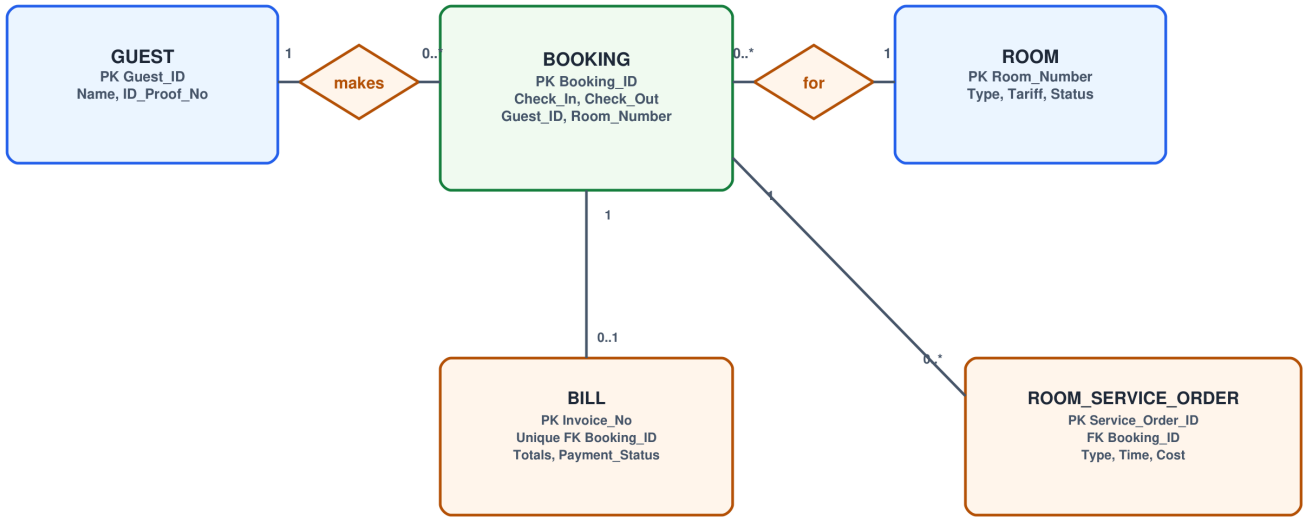
Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.5.1, pp. 61–65; companion Slide 137.

Exercise 3.24. Hotel Reception Desk Management E-R Diagram Architecture

Draw the E–R diagram for a hotel reception desk management system.

Exercise 3.24 — Hotel Reception ER Diagram

Constructed ER model: guests and rooms may have no bookings yet; a bill exists only for a completed stay.



Constructed hotel ER diagram

Core Entities, Attributes, & Relationships:

1. **GUEST Entity:**
   - Primary Key: Guest_ID
   - Attributes: Full_Name, ID_Proof_Type (Passport / National ID), ID_Proof_Number, Address, Contact_Number
2. **ROOM Entity:**
   - Primary Key: Room_Number
   - Attributes: Room_Type (Single, Double, Suite), Daily_Tariff, Floor_Number, Operational_Status (Vacant, Occupied, Cleaning, Maintenance)

3. **BOOKING (Associative Entity):**
    - Bridges GUEST and ROOM across time.
    - Primary Key: Booking_ID; Foreign Keys: Guest_ID, Room_Number
    - Attributes: Check_In_Date, Check_Out_Date, Number_of_Occupants, Advance_Deposit
4. **BILL Entity:**
    - Primary Key: Bill_Invoice_Number
    - Foreign Key: Booking_ID
    - Attributes: Room_Charges_Total, Service_Charges_Total, Taxes_Amount, Discount, Grand_Total, Payment_Status (Paid, Unpaid), Payment_Mode
5. **ROOM_SERVICE Entity:**
    - Primary Key: Service_Order_ID
    - Foreign Key: Booking_ID
    - Attributes: Order_Timestamp, Service_Type (Dining, Laundry), Itemized_Cost

Cardinality Constraints:

- GUEST to BOOKING: A guest has `0..*` bookings; each booking belongs to exactly one guest.
- ROOM to BOOKING: A room has `0..*` bookings over time; each booking reserves exactly one room.
- BOOKING to BILL: A booking has `0..1` finalized bill; each bill belongs to exactly one booking.
- BOOKING to ROOM_SERVICE: One-to-Many (`0..*`) — a booking may accumulate zero or more service charges.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.5.2, pp. 66–68; companion Slide 137.

ملاحظة أكاديمية هامة (Academic Note)

التصميم المعماري للفندق: الخطأ الشائع هو ربط النزيل بالغرفة مباشرة؛ هذا غير صحيح لأن الغرفة يشغلها نزلاء مختلفون عبر الزمن. الكيان الرابط BOOKING هو الذي يحدد من يسكن في أي غرفة ومن أي تاريخ إلى أي تاريخ.

Exercise 3.34. Concrete Construction of an Inconsistent and Incomplete SRS

Construct an example of an inconsistent (incomplete) SRS.

Constructed Example Answer to Exercise 3.34 (Inconsistent and Incomplete SRS):

```markdown [DEFECTIVE SPECIFICATION EXCERPT]
```

Section 2.3: System Safety & Emergency Alert Rules Requirement SAF-01: When patient blood pressure systolic drops below 85 mmHg or pulse exceeds 120 bpm, the bedside monitor shall sound an audible tone of 75 dB and immediately dispatch an urgent network packet to the nursing station.

Section 3.5: Night Shift Operating Modes Requirement NIT-04: Between 22:00 and 06:00, all bedside patient monitor audible tones shall be muted completely to preserve patient sleep; alert messages shall be visual only on the local screen.

Section 4.1: Heart Rate Data Ingestion Requirement HR-02: The software shall sample heart rate readings from sensor port A and display the updated numerical value on screen every 2 seconds.

Section 5.2: Network Outage Recovery Requirement NET-08: The system shall be resilient to network disconnects. ```

Formal Defect Analysis:

1. **Inconsistency (Contradiction) Defect:**
    - SAF-01 mandates that emergency vitals **must sound an audible alarm of 75 dB**.
    - NIT-04 contradicts this by stating that between 22:00 and 06:00 **all audible tones must be muted completely**.
    - *Consequence:* If a patient experiences cardiac collapse at 02:00, the system encounters mutually conflicting requirements: sounding the alarm violates NIT-04, while muting it violates SAF-01.
2. **Incompleteness (Omission) Defect:**
    - In HR-02, the specification defines normal sampling behavior, but **completely omits what happens if sensor port A disconnects, transmits corrupt packet noise, or delivers out-of-range impossible values** (e.g., negative pulse or zero voltage).
    - In NET-08, the statement "*The system shall be resilient to network disconnects*" is completely incomplete: it does not define buffer size, how long data must be retained locally, or the synchronization protocol upon reconnect.

Source for the exercise prompt: K.K. Aggarwal & Yogesh Singh, Exercise 3.34; companion Slide 139. The hospital scenario and its requirements are an authored illustrative example.

📝 **ملاحظة أكاديمية هامة (Academic Note)**

مثال حي للامتحان: تذكر هذا المثال دائماً: التناقض يظهر بين إطلاق جرس الإنذار الطبي وكتم الأصوات ليلاً، بينما النقص يظهر في إهمال سلوك النظام عند انقطاع حساس النبض تماماً.

Section 2: Curriculum Gap-Closing Synthesis (Applied SRS Audit Drill)

Synthesis 09. Synthetic SRS Quality Audit Drill

For each fictional requirement below, identify the quality problem and write a clearer, verifiable version. The examples are authored for practice; they are not quotations or excerpts from student project files.

Practice cases and model answers:

1. **Implementation detail:** *"The system shall be written in Java 17 and use MySQL."* If these technologies are not externally mandated, this constrains design without stating the required system behavior. **Model answer:** *"The system shall store each student's record and retrieve it when an authorized user submits that student's ID."* If a technology is mandated, document it as an approved constraint rather than treating it automatically as a defect.
2. **Ambiguity and verifiability:** *"Search shall be fast and user-friendly."* **Model answer:** *"For the agreed test workload, the system shall return matching records within [approved response-time target] for at least [approved percentage] of searches."* The stakeholders must set the values and define the test workload.
3. **Inconsistency:** *"A student passes if the total is at least 50/100."* A second clause says passing also requires at least 25/40 internally and 30/60 externally. A student with 20 internal and 30 external satisfies the first clause but fails the second. **One possible resolution, if both component minima are intended:** *"A student passes if and only if the internal mark is at least 25/40 and the external mark is at least 30/60."* This is an illustrative policy assumption, not a real university rule.
4. **Omission:** *"The system shall calculate semester GPA when Submit is clicked."* The requirement omits what happens when grades are missing. **Model answer:** *"When Submit is clicked, the system shall check that a mark exists for every enrolled course. If any mark is missing, it shall identify the course and withhold the GPA; otherwise, it shall calculate and display the GPA using the institution-approved formula."*
5. **Unverifiable absolute:** *"The system shall be completely secure and have zero downtime."* **Model answer:** *"The system shall meet the security controls approved for its risk level and maintain at least [stakeholder-approved availability target] over the defined measurement period."* The controls, target, exclusions, and measurement period must be agreed before acceptance testing.

Quality-criteria sources: IEEE Std 830-1998 and K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.6.2, pp. 73–75. The five practice cases are authored examples, not source quotations.

ملاحظة أكاديمية هامة (Academic Note)

مهارة فحص الـSRS الحقيقية: في التدقيق المهني لوثائق الـSRS، ابحث دائماً عن:

1. إقحام أدوات البرمجة وقواعد البيانات في المتطلبات الوظيفية (*Design Infiltration*).
2. الكلمات غير القابلة للقياس مثل "سريع" و"آمن جداً" (*Ambiguity*).
3. النقص عند الحالات الاستثنائية (*Incompleteness*).

ASE Chapter 3

V3 Doctor's Vocabulary Coverage Pack

Term Block 01: Requirements Engineering Process & Present State of Practice

The Requirements Engineering (RE) Process is the disciplined, structured sequence of activities through which customer requirements are discovered, analyzed, formally specified, and validated. The process takes the Problem Statement as input, produces the SRS as output, and consists of four crucial, tightly coupled iterative activities:

Requirements Elicitation: Uncovers stakeholder expectations, operational realities, and system boundaries through collaborative discovery techniques.
Requirements Analysis: Refines, scrutinizes, and models elicited requirements (DFD leveling, Data Dictionary, ER diagrams) to remove ambiguities and resolve conflicts.
Requirements Documentation (SRS): Formalizes requirements into an unambiguous, verifiable contractual document conforming to IEEE Std 830-1998.
Requirements Review (Validation): Rigorously inspects the draft SRS against quality criteria through peer walkthroughs to catch defects early, driving feedback loops.

(Pre-project gateway: A feasibility study evaluates whether the project is viable before significant resources are committed, preceding the four-step requirements-engineering cycle.)

Present State of Requirements Engineering Practice:

The course materials identify recurring practical difficulties:

Requirements are difficult to uncover.
Requirements can change during development.
Teams may rely too heavily on CASE tools.
Project schedules may be tight.
Communication barriers can obstruct understanding.
Market-driven development and limited resources add pressure.

Source: companion Slide 4; K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.1, pp. 40–41.

> 📝 **ملاحظة أكاديمية هامة (Academic Note)**
>
> **ملاحظة الطالب والفخ الامتحاني:**
>
> - لا تسرد مراحل الـSDLC كاملة؛ فخطوات RE المعتمدة في المنهج (شريحة 3 وشكل 3.1) محصورة في: الاستخلاص (Elicitation) ← التحليل (Analysis) ← التوثيق (Documentation) ← المراجعة (Review) مع حلقتي تغذية راجعة. أما دراسة الجدوى فهي تقييم سابق لتحديد جدوى المشروع.
> - Slide 4 يسرد صعوبات الممارسة المذكورة أعلاه؛ لا تحوّل القائمة إلى نتائج استبيان أو ترتيب لأسباب فشل المشاريع.

Sources: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.1–§3.2, pp. 40–45; companion Slides 2–5.

Term Block 02: Types of Requirements & Interface Specifications

Software requirements are categorized into distinct conceptual classes:

Functional Requirements:

- **Statements of services, computations, data transformations, and behavioral responses the system must provide.** They define what the system must do in response to specific operational inputs.

Non-Functional Requirements:

- **Constraints placed on the services or functions offered by the system.** These include performance targets (response time, throughput), quality attributes (reliability, availability, security, usability), platform constraints, and regulatory standards.

User Requirements vs. System Requirements:

- **User Requirements:** High-level abstract statements in natural language and informal diagrams intended for clients and end-users to establish contractual scope.
- **System Requirements:** Detailed, structured technical specifications of all software functions and operational constraints used by software architects and developers for implementation.

Interface Specification (Slide 9 Focus):

- **Crucial for customers and system integrators to ensure seamless interoperability.** Encompasses three primary types:
  - **Procedural Interfaces (Application Programming Interfaces - APIs):** Function signatures, parameters, return types, and exceptions.
  - **Data Structures:** Shared in-memory data representations exchanged across system modules.
  - **Representation of Data:** Standard physical encodings, network packet formats, and communication schemas.

> 📝 **ملاحظة أكاديمية هامة (Academic Note)**
>
> **ملاحظة الطالب والفخ الامتحاني:**
>
> - ميز بدقة بين تصنيف سلايد 9 الخاص بـInterface Types (APIs, Data structures, Representation of data) وبين تبويب معيار IEEE 830 الخارجي (User, Hardware, Software, Communications).
> - المتطلبات الوظيفية تجيب عن: *ماذا يفعل النظام؟* بينما غير الوظيفية تجيب عن: *ما هي قيود الأداء والأمان؟*

Sources: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.3, pp. 45–48; companion Slides 6–9.

Term Block 03: Feasibility Study: Purpose and Focus

Purpose of Feasibility Studies:

A Feasibility Study is an early evaluation and analysis of the potential impact, viability, and practicality of a proposed software project.

Economic Motivation (Slide 10 Historical Note): Companion Slide 10 attributes the figures **31% cancelled, 53% over budget by an average of 189%, and 94 restarts per 100 projects** to a historical IBM report. The slide gives no report date or full citation; treat these as historical course-slide figures, not current universal rates.
Core Principle: A feasibility study checks whether the proposed system is technically, economically, and operationally viable, including whether major risks can be surmounted, before the project proceeds further.

Focus of Feasibility Studies (Slides 13–15):

Product Concept Viability: Is the software concept practical, and can a system matching the vision statement be engineered?
Cost and Schedule Realism: What are the current estimated cost and schedule bounds, and how wide is the gap between original targets and realistic estimates?
Business Model Justification: Is the business model economically justified when realistic costs are factored in?
Stability of Scope: Can users and developers agree on a detailed user interface prototype? If not, requirements are not yet stable.

> 📝 **ملاحظة أكاديمية هامة (Academic Note)**
>
> **ملاحظة الطالب والفخ الامتحاني:**
>
> - دراسة الجدوى ليست مضيعة للوقت بل أداة حماية؛ وإلغاء المشروع غير القابل للتطبيق في هذه المرحلة يعتبر نجاحاً هندسياً وإدارياً يحمي المؤسسة من خسائر فادحة.

Sources: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.2.1, pp. 44–45; companion Slides 10–15.

Term Block 04: Requirements Elicitation: Interviews, Brainstorming, FAST, QFD, & Use Cases

Requirements Elicitation is the proactive discovery, collection, and synthesis of user needs, business workflows, and operational constraints from diverse stakeholders.

1. Interviews (The Foundational Technique):

Mechanism: Direct interpersonal dialogue between systems analysts and domain stakeholders (users, managers, operators) utilizing open-ended and closed questions.
Strengths: Uncovers informal workarounds, builds personal stakeholder rapport, and clarifies complex organizational goals.
Limitations: Subject to cognitive bias; interviewees often omit critical tacit knowledge; extremely time-consuming for large distributed user bases.

2. Brainstorming, FAST, and QFD:

Brainstorming: A group technique for generating and discussing candidate requirements; ideas are collected before the group evaluates and refines them.
FAST (Facilitated Application Specification Technique): A facilitated stakeholder session used to agree on scope, clarify needs, and develop a shared requirements specification.
QFD (Quality Function Deployment): Translates customer needs into prioritized technical requirements, helping teams focus on the product characteristics customers value.

3. The Use Case Approach:

Mechanism: Scenario-based behavioral modeling capturing interactions between external **Actors** and the **System Boundary** to achieve observable user goals.
Stereotype Dependencies:

- **<<include>>: Mandatory, unconditional inclusion** of a shared sub-routine; the base use case cannot complete without it (Base ⟶ Included).
- **<<extend>>: Optional, conditional insertion** of variant behavior triggered only at a defined extension point (Extension ⟶ Base).

> 📝 **ملاحظة أكاديمية هامة (Academic Note)**
>
> **ملاحظة الطالب والفخ الامتحاني:**
>
> - في المقابلات، أكبر عائق هو المعرفة الضمنية (*Tacit Knowledge*) التي يمارسها الموظف كعادة يومية وينسى ذكرها للمحلل.
> - اتجاه الأسهم في Use Case حاسم في الامتحان: <<include>> يشير للأمام نحو الحالة الفرعية الإلزامية، بينما <<extend>> يشير للخلف عائداً إلى الحالة الأصلية.

Sources: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.3.1 & §3.3.4, pp. 48–61; companion Slides 16–26, 27–50.

Term Block 05: Requirements Analysis: DFDs, Data Dictionaries, ERDs, & Prototyping

Requirements Analysis models the operational problem domain to uncover omissions, structural flaws, and boundary conflicts through four primary techniques:

Data Flow Diagrams (DFDs):

- **Hierarchical behavioral modeling tool depicting data movement and functional transformations.**
- Uses four standard symbols: **Process (Bubble), Data Flow (Arrow), Data Store (Parallel Lines), and External Entity (Rectangle).**
- Operates on strict conservation rules: Level-0 Context Diagram must be **strictly balanced** with Level-1 decompositions. Direct connections between external entities or data stores are illegal.

Data Dictionaries (DD):

- **Structured, centralized repository defining the schema, composition, and constraints of every data item in the system.**
- Employs standard formal algebraic operators:
  - = : **is defined as / composed of**
  - + : **AND / Sequence**
  - [a/b] : **select either a or b**
  - y{a} : **repeat a at least y times**
  - {a}z : **repeat a at most z times**
  - y{a}z : **repeat a from y through z times**
  - (a) : **a is optional**

Entity-Relationship Diagrams (ERDs):

- **Structural data modeling depicting persistent data at rest via Entities, Attributes, and Relationships.**
- Identifies relationship degrees: **Unary (Degree 1), Binary (Degree 2), Ternary (Degree 3), and N-ary.**
- When mapping an M:N relationship into relational tables, a junction table is a common implementation option; a conceptual ER diagram may show the M:N relationship directly.

Software Prototyping (Throw-Away vs. Evolutionary):

- **In large software systems, prototyping should be "Throw-Away".** Developed using "quick and dirty" methods to clarify requirements; discarding the experimental code prevents unmodular, unmaintainable artifacts from contaminating production architecture.

> 📝 **ملاحظة أكاديمية هامة (Academic Note)**
>
> **ملاحظة الطالب والفخ الامتحاني:**
>
> - الـDFD يمثل حركة العمليات، والـERD يمثل بنية البيانات، والقاموس يوحّد معنى عناصر تدفق البيانات.
> - الكتاب يستخدم [a/b] للاختيار، بينما تظهر صيغة مختلفة في Slide 88؛ اعتمدت هنا صيغة الكتاب الأساسي.
> - النموذج التجريبي السريع (*Throw-away prototype*) صُمم لجمع المعرفة والرمي في القمامة؛ ومحاولة بيعه أو تحويله لمنتج حقيقي هي بداية الكارثة البرمجية.

Sources: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.5, pp. 61–72; companion Slides 82–91, 116.

Term Block 06: Requirements Documentation: Nature, Characteristics, & IEEE Std 830

1. Nature of the Software Requirements Specification (SRS):

The SRS is the official, formal document that establishes the binding contractual agreement between customer and development organization.
Black-Box Abstraction Principle: It completely specifies external behavior, inputs, outputs, and constraints **without prescribing internal design, algorithms, or code architecture.**
Core Issues Addressed: Functionality, External Interfaces, Performance, Quality Attributes, and Design Constraints.

2. Characteristics of a Good SRS (IEEE Std 830-1998 Criteria):

Correct: Accurately reflects true customer intentions.
Unambiguous: Every requirement statement has exactly one valid interpretation.
Complete: Defines responses to all possible input conditions (valid and invalid).
Consistent: Contains no conflicting requirements or mutually contradictory states.
Ranked for Importance/Stability: Prioritizes features into essential, conditional, and optional.
Verifiable: A finite, cost-effective test or inspection process exists to prove conformance.
Modifiable: Structured modularly so changes do not produce uncontained ripple effects.
Traceable: Every requirement possesses a unique tag supporting forward and backward tracing.

3. Organization of the SRS (IEEE Std 830 3-Tier Structure):

Section 1: Introduction: Purpose, Scope, Definitions/Acronyms, References, and Overview.
Section 2: Overall Description: Product Perspective, Functions Summary, User Characteristics, General Constraints, and Dependencies.
Section 3: Specific Requirements: Detailed verifiable functional rules, interface specifications (User, Hardware, Software, Communications), and performance bounds.

> 📝 **ملاحظة أكاديمية هامة (Academic Note)**
>
> **ملاحظة الطالب والفخ الامتحاني:**
>
> - الشرط الأكثر تكراراً في الامتحانات هو **Verifiable**؛ أي متطلب يحتوي على صفات غير مقاسة مثل "سريع" أو "واجهة سهلة" يفقد صفة التحقق ويعد عيباً في الـSRS.
> - تذكر دائماً الأقسام الثلاثة الرئيسية لمعيار IEEE 830: المقدمة، الوصف العام، والمتطلبات المحددة.

Sources: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.6, pp. 72–78; companion Slides 101–110.

Term Block 07: Requirements Validation: The 6-Stage Review Process & Checklists

Requirements Validation ensures that the SRS accurately represents customer expectations and satisfies institutional quality standards before architectural design commences.

1. The 6-Stage Requirements Review Process (Figure 3.21 & Slide 113):

Plan Review: Establish review scope, schedule sessions, and select the multi-disciplinary inspection team.
Distribute SRS Documents: Deliver the draft specification to reviewers in advance.
Read Documents: Reviewers individually inspect the document, noting ambiguities and standard violations.
Organize Review: Convene structured walkthrough to log defects and consensus resolutions.
Follow-Up Actions: Remediate logged issues, consult stakeholders on conflicts, and verify needed changes.
Revise Document: Requirements authors modify the SRS to produce the approved baseline document.

2. Problem Resolution Actions (Slide 114):

Requirements Clarification: Rewrite vague or subjective phrasing.
Missing Information: Re-engage stakeholders to supply omitted operational rules.
Requirements Conflicts: Convene stakeholder negotiations to resolve contradictory demands.
Unrealistic Requirements: Consult management to de-scope technically or economically infeasible items.
Security Issues: Align system requirements with mandatory security standards.

3. The 7-Point Quality Review Checklist (Slide 115):

1. **Redundancy:** Are statements repeated unnecessarily?
2. **Completeness:** Are all inputs, outputs, and operational states defined?
3. **Ambiguity:** Can any requirement statement be interpreted in multiple ways?
4. **Consistency:** Are there conflicting constraints or contradictory states?
5. **Organization:** Does the document strictly conform to the IEEE Std 830 structure?
6. **Conformance:** Does the specification adhere to institutional and legal standards?
7. **Traceability:** Is every requirement uniquely tagged and bidirectionally traceable?

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

- مراجعة المتطلبات (*Requirements Review*) عملية جماعية منظمة وليست مجرد قراءة سريعة؛ والخطوة التي تفشل فيها معظم الفرق هي إهمال القراءة الفردية المسبقة (الخطوة 3).
- تسلسل الشريحة 113 يوجه من تنظيم المراجعة (Organize review) إلى إجراءات المتابعة (Follow up actions) ثم تعديل الوثيقة (Revise document). احفظ نقاط الفحص السبعة نصاً كما وردت في سلايد 115.

Sources: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.7, pp. 92–94; companion Slides 112–115.
